

ตัวอย่าง การดำเนินการตามขั้นตอนของ Engineering Design Process ของวิชา CEPP 2568

Case Study (สมมติ)

บริษัท WorkHub Co., Ltd. เป็นผู้ให้บริการพื้นที่ทำ งานร่วม (Co-working Space) ในย่าน ธุรกิจใจกลาง กรุงเทพมหานคร ปัจจุบันมีการขยายสาขาอย่างรวดเร็วและมีสมาชิกกว่า 500 ราย ทำให้การบริหารจัดการด้วยระบบ แบบ Manual เริ่มไม่เพียงพอต่อความต้องการ

ปัญหาที่พบในปัจจุบัน (Pain Points)

- การจองซ้ำซ้อน - ไม่มีระบบตรวจสอบ real-time ทำให้คนหลายคนจองพื้นที่เดียวกัน
- ปัญหา No-show - สมาชิกจองแล้วไม่มาใช้งาน ทำให้เสียรายได้และกระทบคนอื่น
- การคิดค่าบริการซับซ้อน - เพราะมีหลายระดับสมาชิก หลายสิทธิประโยชน์
- การขอทรัพยากรเพิ่มยุ่งยาก - เช่นต้องการจองเพิ่มต้องติดต่อเจ้าหน้าที่
- ไม่มีระบบติดตามประวัติ - ทั้งการใช้งานและการชำระเงิน

ข้อมูลผู้ใช้

- สมาชิกทั่วไป - จองได้ 14 วันล่วงหน้า, มีการจองพร้อมกันได้ไม่เกิน 3 รายการ
- สมาชิกพรีเมียม (Silver/Gold/Platinum) - จองได้ 30 วันล่วงหน้า, ได้ส่วนลด 5-15%, มีชั่วโมงฟรี
- เจ้าหน้าที่สาขา - Check-in/out, จัดการจอง, รับชำระเงิน
- ผู้จัดการระบบ - จัดการข้อมูลหลัก, อนุมัติค่าปรับ, ดูรายงานทุกสาขา

ประเภทพื้นที่

- Hot Desk (โต๊ะไม่ประจำ) - 80 บาท/ชั่วโมง
- Dedicated Desk (โต๊ะประจำมีล็อกเกอร์) - 120 บาท/ชั่วโมง
- Private Office (ห้องส่วนตัว 2-4 คน) - 350 บาท/ชั่วโมง
- Phone Booth (ห้องโทรศัพท์) - 50 บาท/30 นาที

ห้องประชุม 4 ขนาด:

- Small (4 คน) - 200 บาท/ชั่วโมง
- Medium (8 คน) - 400 บาท/ชั่วโมง
- Large (16 คน) - 800 บาท/ชั่วโมง
- Training (30 คน) - 1,500 บาท/ชั่วโมง

ทรัพยากรเสริม: จอมอนิเตอร์, ชุดประชุมทางไกล, ไวท์บอร์ด, โต๊ะปรับระดับ, ล็อกเกอร์

ขั้นตอนที่ 1 (Discovery → Stakeholder)

ขั้นตอนที่ 1.1 ระบุและจัดประเภทผู้มีส่วนได้ส่วนเสีย (Stakeholder Identification & Classification)

รายชื่อผู้มีส่วนได้ส่วนเสียทั้งหมด

A) ผู้ใช้ปลายทาง (End Users)

1. สมาชิกทั่วไป (Regular Member) – ผู้จอง/ใช้งานหลัก
2. สมาชิกพรีเมียม (Premium Member: Silver/Gold/Platinum) – เงื่อนไข/สิทธิแตกต่าง
3. ลูกค้าองค์กร (Corporate Member/กลุ่มผู้ใช้หลายคน) – เงื่อนไขสัญญา/ส่วนลด/การชำระเงินต่างจากรายบุคคล

B) ผู้ปฏิบัติการหน้างาน (Operations)

4. เจ้าหน้าที่ประจำสาขา (Branch Staff)
5. ผู้จัดการสาขา/หัวหน้าทีมปฏิบัติการ (Branch Manager/Operations Lead)

C) ผู้บริหารระบบและกำกับนโยบาย (Governance & Control)

6. ผู้ดูแลระบบ (System Administrator)
7. ผู้บริหารบริษัท/ผู้สนับสนุนโครงการ (Executive Sponsor: CEO/COO)

D) การเงิน/บัญชี/กฎหมาย (Business Support)

8. ทีมการเงิน/บัญชี (Finance & Accounting)
9. ทีมกฎหมาย/Compliance/PDPA

E) ผู้ให้บริการภายนอก (External Partners / Integrations)

10. ผู้ให้บริการ Payment Gateway (เช่น Omise/2C2P)
11. ผู้ให้บริการยืนยันตัวตน (Google/LINE Login, 2FA)
12. ระบบปฏิทิน (Google/Outlook Calendar)

F) ผู้ได้รับผลกระทบทางอ้อม

13. สมาชิก “ที่นั่งข้างเคียง/ผู้จองไม่ทัน”
14. ทีมดูแลทรัพยากร/แม่บ้าน/ซ่อมบำรุง

ขั้นตอนที่ 1.2 แผนที่ผู้มีส่วนได้ส่วนเสีย (Stakeholder Mapping: Influence–Interest–Impact)

1) นิยามระดับ (ใช้มาตรฐานเดียวกันทั้งทีม)

- Influence (อิทธิพล): สูง/กลาง/ต่ำ
- Interest (ความสนใจ): สูง/ต่ำ
- Impact (ผลกระทบ): สูง/กลาง/ต่ำ

2) Stakeholder Mapping

กลุ่ม	Stakeholder	Influence	Interest	Impact	เหตุผลสั้นๆ (โยงกับระบบ)
Governance	Executive Sponsor	สูง	สูง	กลาง	ชี้เป้า KPI/งบ/ทิศทางผลลัพธ์ธุรกิจ (เช่น ลด no-show, ลดซ้ำซ้อน)
Governance	System Administrator	สูง	สูง	สูง	ตั้ง policy/ข้อมูลหลัก/อนุมัติ refund/ยกเลิกค่าปรับ
Ops	Branch Manager	กลาง-สูง	สูง	สูง	รับผิดชอบ SLA หน่วยงาน, ต้องจัดการข้อบกพร่อง/การให้บริการ
Ops	Branch Staff	กลาง	สูง	สูง	ใช้งานทุกวัน (check-in/out, walk-in, รับชำระเงิน)
End Users	Premium Member	ต่ำ-กลาง	สูง	สูง	กฎ/สิทธิซับซ้อน (quota/เครดิต/ส่วนลด) ถ้าระบบผิดกระทบมาก
End Users	Regular Member	ต่ำ	สูง	สูง	ผู้ใช้งานจำนวนมาก เจอ pain point จองซ้ำซ้อน/โน้ว/ชำระเงิน
Support	Finance & Accounting	กลาง	กลาง	กลาง	ต้องการความถูกต้อง billing, reconciliation, รายงานรายได้
Compliance	Legal/PDPA	กลาง	ต่ำ-กลาง	กลาง	ความเสี่ยงข้อมูลส่วนบุคคล/การเข้าถึง/การเงิน
External	Payment Gateway	กลาง	ต่ำ	กลาง	constraint ทางเทคนิค/Refund/e-receipt
External	Auth Provider	กลาง	ต่ำ	กลาง	ข้อกำหนด login/2FA
External	Calendar Provider	ต่ำ	ต่ำ	ต่ำ-กลาง	Integration เพิ่มความสะดวก แต่ไม่ใช่แกนรายได้
Indirect	Maintenance/Resource team	ต่ำ-กลาง	กลาง	กลาง	สถานะซ่อมบำรุง/ทรัพยากรชำรุด/กระบวนการส่งมอบ

3) จัดกลุ่มลง “เมทริกซ์ Influence-Interest” เพื่อกำหนดวิธีจัดการ

- **Manage Closely (อิทธิพลสูง + สนใจสูง):** Executive Sponsor, System Admin, Branch Manager
- **Keep Satisfied (อิทธิพลสูง + สนใจต่ำ):** Legal/PDPA (บางช่วง), Finance (บางช่วงที่ไม่ใช่ UAT)
- **Keep Informed (อิทธิพลต่ำ + สนใจสูง):** Members (Regular/Premium/Corporate), Branch Staff
- **Monitor (อิทธิพลต่ำ + สนใจต่ำ):** Calendar Provider (ช่วงแรก), บางกลุ่มผลกระทบทางอ้อม

ขั้นตอนที่ 1.3 กลยุทธ์การสื่อสารและการมีส่วนร่วม (Communication & Engagement Strategy)

A) Manage Closely — (Sponsor, Admin, Branch Manager)

เป้าหมาย: ตรึงเป้าหมาย/นโยบาย/ข้อยกเว้นให้ชัดเจนก่อนออกแบบ

- **ช่องทาง:** ประชุมสั้นรายสัปดาห์ (30 นาที) + สรุป 1 หน้า
- **สิ่งที่ต้องได้จากเขา:**
 1. KPI/Target ที่ต้องบรรลุ (เช่น ลด no-show, ลด booking conflict, ลดเวลาคิดเงิน)

coworking_space_assignment.docx

2. Policy ที่ “ยอมรับได้” เช่น กฎ no-show, ยกเลิกซ้ำ, วงเงินค้างชำระ

coworking_space_assignment.docx

- **Artifact ที่ส่ง:** Project Charter (1 หน้า), Decision Log (รายการตัดสินใจ), Policy Draft

B) Keep Informed — (Members, Branch Staff)

เป้าหมาย: เก็บ VoC/Pain point + ทดสอบต้นแบบการใช้งาน (UAT) ให้ครอบคลุม “งานจริง”

- **ช่องทาง:** สัมภาษณ์/แบบสอบถาม + Prototype walkthrough
- **ประเด็นถามสมาชิก:**
 - ปัญหาจองซ้ำซ้อน/ไม่เห็นสถานะ real-time

coworking_space_assignment.docx

- ประสิทธิภาพโดน no-show กระทบ / ความยุติธรรมของค่าปรับ

coworking_space_assignment.docx

- ความคาดหวัง self-service upgrade ทรัพยากร

coworking_space_assignment.docx

- **ประเด็นถามพนักงานสาขา:**

- จุดติดขัด check-in/out, walk-in, คืนทรัพยากร, รับชำระเงิน

coworking_space_assignment.docx

- **Artifact ที่ส่ง:** Journey Map (สมาชิก/พนักงาน), Issue List (จัดลำดับความสำคัญ)

C) Keep Satisfied — (Finance, Legal/PDPA, Payment Provider)

เป้าหมาย: ไม่ให้ติดข้อจำกัดสำคัญช่วงท้าย

- **ช่องทาง:** Review เป็นรอบ (milestone) ไม่ต้องถี่ แต่ต้อง “ชัดเจน”
- **จุดตรวจ (Checkpoints):**

- Finance: โครงสร้างค่าบริการ/ส่วนลด/ตัวอย่างการคำนวณ

coworking_space_assignment.docx

- Legal/PDPA: การปกป้องข้อมูลส่วนบุคคล/สิทธิการเข้าถึง/การเก็บ log

coworking_space_assignment.docx

- Payment: flow การชำระเงิน/คืนเงิน/ใบเสร็จ

coworking_space_assignment.docx

- **Artifact ที่ส่ง:** Data Dictionary (ข้อมูลสำคัญ), Access Control Matrix, Billing Rules Spec

D) Monitor — (Calendar integration, stakeholder ผลกระทบต่ำในระยะแรก)

เป้าหมาย: ไม่เสียเวลาเกินจำเป็นช่วง MVP

- **ช่องทาง:** แจ้งความคืบหน้าเป็นครั้งคราว / ทำเมื่อ core พร้อม
- **Artifact:** Integration backlog (ทำหลัง core booking+billing เสร็จ)

ผลลัพธ์ที่ “ควรมี” เมื่อจบขั้นตอนนี้ (Deliverables)

1. รายชื่อ stakeholder ครบ + จัดกลุ่ม (End user/Ops/Governance/Support/External)
2. ตาราง Stakeholder Mapping (Influence/Interest/Impact) + สรุป 4-quadrant strategy
3. แผนสื่อสารและการมีส่วนร่วมรายกลุ่ม (ช่องทาง/ความถี่/สิ่งที่ต้องได้/เอกสารที่ส่ง)

ขั้นตอนที่ 2 : การค้นพบความต้องการของลูกค้า (Voice of the Customer – VoC)

วัตถุประสงค์ของขั้นตอนนี้

1. เข้าใจ Pain Point ที่แท้จริง ของผู้ใช้
2. ค้นหา ความต้องการที่ซ่อนอยู่ (Underlying Wants / Root Cause)
3. แปลงความต้องการเชิงคุณภาพ → แนวทางออกแบบเชิงวิศวกรรมในขั้นถัดไป
4. ให้นักศึกษา Fully Internalize the VOC ก่อนคิดโซลูชัน

ขั้นตอนที่ 2.1 การเลือก Stakeholder สำหรับการสัมภาษณ์ (Sample Selection)

Stakeholder ที่เลือก (ตัวอย่างหลัก) : Premium Member (Gold Member)

เหตุผล:

- ใช้งานบ่อย
- ใช้ทรัพยากรหลากหลาย (โต๊ะ, ห้องประชุม, อุปกรณ์)
- ได้รับผลกระทบจากระบบมากที่สุด
- มีความคาดหวังสูง (Critical User)

ขั้นตอนที่ 2.2 การตั้งคำถาม (ก่อนเริ่มสัมภาษณ์)



❌ ไม่ถามว่า “อยากได้ระบบแบบไหน”



✅ ถามว่า “ปัจจุบันคุณเจอปัญหาอะไร / รู้สึกอย่างไร / ทำอย่างไรอยู่”

โครงคำถามจะไหลตามลำดับ:

1. ประสบการณ์จริง
2. ความไม่สะดวก
3. ผลกระทบ (เวลา / อารมณ์ / เงิน)
4. วิธีที่เขาใช้แก้ปัญหาเอง
5. ความคาดหวัง (โดยยังไม่พูดถึง solution)

ขั้นตอนที่ 2.3 การสัมภาษณ์ VoC (ตัวอย่างบทสนทนาจริง)

ช่วงที่ 1 : การใช้งานจริง (Context & Experience)

Q1: “ปกติคุณใช้ coworking space นี้บ่อยแค่ไหน และใช้ทำอะไรเป็นหลักครับ?”

A1: “ผมมาประมาณอาทิตย์ละ 4-5 วันครับ ส่วนใหญ่ใช้โต๊ะทำงาน แต่ก็มีห้องประชุมเล็กบ้าง เวลาวิวงานกับลูกค้า หรือคุยกับทีม remote”

การตีความ (ยังไม่วิเคราะห์ลึก)

- ผู้ใช้ heavy user
- ใช้หลาย resource → ความซับซ้อนสูง
- ถ้าระบบมีปัญหา จะกระทบเขามาก

ช่วงที่ 2 : การเริ่มเจอปัญหา (Surface Pain Point)

Q2: “ถ้าให้นึกถึงปัญหาที่เจอบ่อยที่สุดเกี่ยวกับการจองหรือใช้งานพื้นที่ คืออะไรครับ?”

A2: “ที่เจอบ่อยสุดคือ เรื่องการจองครับ บางทีผมจองห้องไว้แล้ว แต่พอมาถึงมันเหมือนมีปัญหา เช่น มีคนอื่นใช้อยู่ หรือสถานะในระบบไม่ตรงกับของจริง”

การวิเคราะห์ระดับ Problem / Issue

- ❌ ไม่ใช่แค่ “ระบบไม่ดี”
- ✓ ปัญหา = ข้อมูลสถานะไม่ตรงกับความจริง (Data Inconsistency)

ช่วงที่ 3 : ผลกระทบทางอารมณ์และเวลา (Impact)

Q3: “สถานการณ์แบบนี้ส่งผลกับคุณอย่างไรบ้างครับ?”

A3: “มันเสียเวลามากครับ แล้วก็ค่อนข้างเครียด เพราะบางทีนัดลูกค้าไว้แล้ว ต้องมานั่งรอ หรือเดินหาพนักงานให้ช่วยจัดการให้ใหม่ ดูไม่เป็นมืออาชีพเลย”

การวิเคราะห์ สิ่งที่ถูกคำพูด:

“เสียเวลา”, “เครียด”, “ดูไม่เป็นมืออาชีพ”

Underlying Impact

- เสียความน่าเชื่อถือกับลูกค้า
- ความเครียดสูงในสถานการณ์ที่ควร “ราบรื่น”

ช่วงที่ 4 : เจาะลึกด้วย “ทำไม” (Root Cause)

Q4: “ถ้าลองมองลึกขึ้น คุณคิดว่าทำไมปัญหานี้ถึงเกิดซ้ำ ๆ ครับ?”

A4: “ผมรู้สึกว่าการระบบมันไม่ค่อย real-time ครับ บางทีมีคน walk-in ใช้ก่อน หรือมีคนจองแล้วไม่มา แต่สถานะไม่อัปเดต ทำให้ทุกอย่างมันมั่วไปหมด”

วิเคราะห์ Root Cause

- ✗ ไม่ใช่ปัญหาพฤติกรรมผู้ใช้ล้วน ๆ
- ✓ ปัญหาที่แท้จริงคือ

การจัดการสถานะทรัพยากรแบบ Real-Time และการจัดการ No-show

ช่วงที่ 5 : วิธีที่ลูกค้า “แก้ปัญหเอง” (Workaround)

Q5: “แล้วเวลาที่เจอปัญหาแบบนี้ คุณจัดการอย่างไรครับ?”

A5: “ส่วนใหญ่ก็ต้องไปคุยกับพนักงานครับ บางครั้งก็ยอมเปลี่ยนแผน ไม่จองล่วงหน้า ใช้เอานั่งานเลย จะได้ไม่ต้องมาลุ้น”

Insight สำคัญ

⚠️ ลูกค้ายอม เลิกใช้ฟีเจอร์จองล่วงหน้า

เพราะ ไม่เชื่อมั่นในระบบ

2.4 การสรุป VoC ด้วย VOC Translation Matrix

Raw Customer Comment	Problem / Issue	Underlying Want (Root Cause)
“จองแล้วแต่มีคนไข้อยู่”	สถานะทรัพยากรไม่ตรงจริง	ความถูกต้องของข้อมูลแบบ Real-Time
“เสียเวลา เครียด ดูไม่เป็นมืออาชีพ”	ระบบไม่เสถียร	ความเชื่อมั่น (Trust) ในระบบ
“ไม่กล้าจองล่วงหน้าแล้ว”	ผู้ใช้หลีกเลี่ยงฟีเจอร์	ระบบต้อง ‘พึงพาได้’
“อยากรู้สถานะชัดเจน”	ข้อมูลไม่โปร่งใส	ความชัดเจนและคาดการณ์ได้

2.5 การ Internalize VOC (สิ่งที่นักศึกษาต้อง “เข้าใจให้ลึก”)

 Pain Point ที่แท้จริงไม่ใช่ “ระบบจอง”

แต่คือ:

 ความต้องการที่แท้จริง (Fully Internalized VOC)

1. ความเชื่อมั่น (Trustworthiness)

ผู้ใช้ต้อง “กล้าพึ่งระบบ” ในสถานการณ์จริง

2. ความถูกต้องแบบ Real-Time (Data Accuracy)

สถานะทรัพยากรต้องตรงกับโลกจริง

3. การลดความเครียดและความไม่แน่นอน (Stress Reduction)

ผู้ใช้ไม่ควรต้องมี Plan B ตลอดเวลา

ผลลัพธ์ของขั้น VoC (Deliverables)

เมื่อจบขั้นนี้ นักศึกษาควรมี:

1. บันทึกบทสัมภาษณ์จริง
2. VOC Translation Matrix ที่ครบทุกช่อง
3. ความเข้าใจเชิงลึกกว่า
“เรากำลังแก้ปัญหาอะไรให้ใคร และทำไมมันสำคัญ”

ขั้นตอนที่ 3 การนิยามปัญหาอย่างเข้มงวด (Define the Problem)

วัตถุประสงค์ของขั้นตอน Define

ขั้นตอนนี้มีหน้าที่ “แปลงเสียงของลูกค้า (VoC)” ให้กลายเป็น ข้อความปัญหาที่ชัดเจน วัดผลได้ และพร้อมเข้า

สู่การออกแบบเชิงวิศวกรรม

เมื่อจบขั้นตอนนี้ นักศึกษาต้องตอบได้อย่างมั่นใจว่า:

-  เรากำลังแก้ปัญหาอะไร “จริง ๆ”
-  ปัญหานี้สำคัญกับใคร
-  ความสำเร็จของการแก้ปัญหาคืออะไร (ในเชิงพฤติกรรมและเชิงระบบ)

3.1 การค้นหารากเหง้าของปัญหา (Root Cause Analysis – Five Whys)

3.1.1 ปัญหาที่สังเคราะห์จาก VoC (Problem Symptom)

อาการของปัญหา (Symptom):

สมาชิกระดับ Premium ที่ใช้ coworking space เป็นประจำ ประสบปัญหา “จองห้องประชุมล่วงหน้าแล้วไม่สามารถใช้งานได้จริง” เนื่องจากสถานะทรัพยากรในระบบไม่ตรงกับสภาพการใช้งานจริง

3.1.2 การวิเคราะห์ Five Whys (ทีละชั้น อย่างมีเหตุผล)

Why #1: ทำไมสมาชิกจองแล้วแต่ใช้งานไม่ได้?

→ เพราะเมื่อมาถึง พบว่าห้องประชุมมีผู้อื่นใช้งานอยู่ หรือถูกใช้แบบ walk-in

Why #2: ทำไมระบบถึงแสดงว่าว่าง แต่ความจริงไม่ว่าง?

→ เพราะสถานะทรัพยากรในระบบไม่ถูกอัปเดตแบบ Real-Time

Why #3: ทำไมสถานะทรัพยากรถึงไม่ Real-Time?

→ เพราะการใช้งานจริง (walk-in, no-show, การยกเลิกหน้างาน) ไม่ได้ถูกเชื่อมโยงและบังคับให้สะท้อนกลับเข้าสู่ระบบทันที

Why #4: ทำไมระบบถึงไม่สามารถจัดการเหตุการณ์เหล่านี้ได้อย่างอัตโนมัติ?

→ เพราะระบบถูกออกแบบมาเพื่อ “บันทึกการจอง” ไม่ใช่เพื่อ “ควบคุมความถูกต้องของสถานะทรัพยากร”

Why #5: ทำไมปัญหานี้จึงสำคัญในเชิงวิศวกรรม?

→ เพราะความไม่ถูกต้องของข้อมูลสถานะ ทำให้ผู้ใช้ สูญเสียความเชื่อมั่น (Trust) ต่อระบบ และเลือกหลีกเลี่ยงการใช้งานฟีเจอร์หลักของแพลตฟอร์ม

3.1.3 Root Cause ที่แท้จริง (ไม่ใช่อาการ)



รากเหง้าของปัญหา (Root Cause): ระบบขาดกลไกเชิงวิศวกรรมในการ ควบคุมและรักษาความถูกต้องของสถานะทรัพยากรแบบ Real-Time ภายใต้เหตุการณ์ใช้งานจริงที่เปลี่ยนแปลงตลอดเวลา

3.2 การสร้างข้อความปัญหา (Point of View – POV)

3.2.1 โครงสร้าง POV ที่ใช้

[ใคร] ต้องการ [อะไร] เพื่อ [เป้าหมาย/คุณค่า] เพราะ [เหตุผลเชิงลึก]

3.2.2 POV Statement

สมาชิกระดับ Premium ของ coworking space ต้องการ ระบบบริหารจัดการการจองและสถานะทรัพยากรที่เชื่อถือได้ ซึ่งสามารถ สะท้อนสถานะการใช้งานจริงของพื้นที่และห้องประชุมแบบ Real-Time เพื่อให้พวกเขา สามารถวางแผนการทำงานและนัดหมายลูกค้าได้อย่างมั่นใจ โดยไม่ต้องเตรียมแผนสำรองหรือพึ่งพาการแก้ปัญหาเฉพาะหน้าจากพนักงาน

3.2.3 เหตุผลที่ POV นี้ “ดีในเชิงวิศวกรรม”

- ระบุ ผู้ใช้เป้าหมายชัดเจน (Premium Member)
- ไม่ผูกติดกับ solution ใด ๆ
- เชื่อมโยงโดยตรงกับ **คุณค่าที่วัดผลได้** (Trust, Reliability)
- พร้อมนำไปแตกเป็นข้อกำหนดเชิงเทคนิค

3.3 การแปลง POV เป็นคำถาม “How Might We” (HMW)

หลักการ

HMW ต้อง:

- เปิดกว้างพอให้สร้างสรรค์
- แคบพอที่จะนำไปออกแบบเชิงวิศวกรรมได้
- เชื่อมโยงกับ Root Cause โดยตรง

3.3.1 ชุดคำถาม HMW (แตกตามมุมวิศวกรรม)

HMW #1 – มุมความถูกต้องของข้อมูล (Data Integrity)

เราจะทำอย่างไรได้บ้าง ที่จะทำให้สถานะห้องประชุมและทรัพยากร สะท้อนการใช้งานจริงได้อย่างถูกต้องและทันเวลา แม้จะมีเหตุการณ์ walk-in, no-show หรือการเปลี่ยนแปลงหน้างาน?

HMW #2 – มุมความเชื่อมั่นของผู้ใช้ (User Trust)

เราจะทำอย่างไรได้บ้าง ที่จะทำให้ผู้ใช้ “กล้าพึ่งพา” ระบบการจอง โดยไม่ต้องเตรียมแผนสำรองหรือสอบถามพนักงานซ้ำ?

HMW #3 – มุมการจัดการเชิงระบบ (System Control)

เราจะทำอย่างไรได้บ้าง ที่จะออกแบบระบบให้สามารถ ควบคุมและแก้ไขความคลาดเคลื่อนของสถานะทรัพยากรได้โดยอัตโนมัติ ภายใต้ข้อจำกัดการใช้งานจริงของ coworking space?

3.4 สรุปผลลัพธ์ของขั้น Define (Define Output Summary)

เมื่อจบขั้นตอน Define นี้ ทีมโครงการจะได้:

1.  Root Cause ที่แท้จริง (ไม่ใช่แค่อาการ)
2.  POV Statement ที่ชัดเจนและ actionable
3.  ชุด HMW ที่พร้อมนำไปสู่การสร้างแนวคิด (Ideation)
4.  ขอบเขตปัญหาที่ “คุมได้” ในระดับ Capstone

 จุดสำคัญสำหรับนักศึกษา หากยังตอบ HMW ไม่ได้ แสดงว่า “ยังเข้าใจปัญหาไม่ลึกพอ”

ขั้นตอนที่ 4: การแปลง VoC → ข้อกำหนดที่วัดผลได้ (Quantifiable Requirements / CTQs)

VoC คือ “คำพูด” ของลูกค้า เช่น “เครียด”, “ระบบไม่น่าเชื่อ”, “จองแล้วใช้ไม่ได้”

แต่โครงการวิศวกรรมต้องตอบด้วย ตัวเลข + วิธีพิสูจน์ (verification)

ดังนั้น Output ที่ต้องได้เมื่อจบขั้นตอนนี้คือ:

1. รายการ Quantifiable Requirements (QRs/CTQs) ที่เป็นตัวเลข (มีหน่วย มีเงื่อนไขทดสอบ)
2. ระบุว่าแต่ละ QR จะ วัดอย่างไร ทดสอบอย่างไร ผ่าน/ไม่ผ่าน
3. พร้อมนำไปทำ QFD/HoQ และแปลงต่อเป็น ECs ในขั้น 5

4.1 วิธีทำแบบเป็นขั้นบันได (Algorithm ที่ให้นักศึกษาทำตามได้)

Step 4.1.1 รวบรวม “Raw Customer Comments” (คำพูดจริง)

จากบทสัมภาษณ์ Premium Member (คุณธนา) เรามีตัวอย่างประโยคเช่น:

- “จองแล้วแต่มีคนไข้อยู่”
- “สถานะไม่ real-time”
- “เสียเวลา เครียด คู่มือไม่เป็นมืออาชีพ”
- “สุดท้ายไม่กล้าจองล่วงหน้าแล้ว”
- “อยากให้เชื่อถือได้ จองแล้วได้ใช้จริง”

กฎสำคัญ: ห้ามเริ่มคิดตัวเลขทันที ต้อง “แปล” คำพูดเป็น “ปัญหา/ความต้องการ” ก่อน

Step 4.1.2 แปลเป็น Problem/Issues to be solved (สิ่งที่ต้องแก้ “ในระบบ”)

ตัวอย่าง:

- “จองแล้วแต่มีคนไข้อยู่” → Booking conflict / Overbooking / Status mismatch
- “ไม่ real-time” → ข้อมูลสถานะทรัพยากรไม่ทันเหตุการณ์
- “ต้องเดินหาพนักงาน” → ต้องพึ่ง manual intervention สูง

นี่คือคอลัมน์ 2 ใน VOC Translation Matrix

Step 4.1.3 ขุดหา Underlying Wants (ทำไมลูกค้าถึงพูดแบบนั้น)

ให้ใช้คำถาม: ทำไมเขาถึงพูดแบบนี้? เขาต้องการอะไรที่ลึกกว่านั้น?

ตัวอย่าง Underlying Wants:

- Trust / Reliability: จองแล้ว “ต้องได้ใช้จริง”
- Real-time accuracy: สถานะต้องตรงกับโลกจริง
- Predictability / ลดความไม่แน่นอน: ไม่ต้องมี Plan B
- Speed of resolution: ถ้ามีเหตุการณ์ผิดปกติ ต้องจัดการเร็ว

Step 4.1.4 “สร้าง QRs/CTQs” (ทำให้เป็นตัวเลขที่พิสูจน์ได้)

กฎหลักในการเขียน QR: QR ที่ดีต้องตอบให้ครบ 6 ช่องนี้เสมอ

- 1. Metric ชื่ออะไร (วัดอะไร)
- 2. Unit หน่วยอะไร
- 3. Target ตัวเลขเท่าไร (เป้าหมาย)
- 4. Test condition วัดในเงื่อนไขไหน (ช่วงเวลา/โหนด/สถานการณ์)
- 5. Method วัดอย่างไร (เครื่องมือ/ข้อมูล/ขั้นตอน)
- 6. Pass/Fail เกณฑ์ผ่านคืออะไร

4.2 VOC Translation Matrix (เวอร์ชัน “ตัวอย่างเต็ม” สำหรับ coworking)

Raw Customer Comment (คำพูดจริง)	Problem/Issue ที่ต้องแก้	Underlying Wants (ความต้องการที่ซ่อนอยู่)	Quantifiable Requirement (QR/CTQ) ที่เสนอ
“จองแล้วแต่มีคนใช้อยู่”	booking conflict / สถานะห้องไม่ตรงจริง	Trust + Real-time accuracy	QR1 Booking Fulfillment Rate: สัดส่วนการจองที่ “ใช้งานได้จริง” $\geq 99\%$
“สถานะไม่ real-time”	อัปเดตสถานะล่าช้า	Real-time accuracy	QR2 Status Update Latency: เวลาในการอัปเดตสถานะหลังเหตุการณ์หน้างาน ≤ 5 วินาที
“เสียเวลา ต้องเดินหาพนักงาน”	ต้องใช้ manual intervention สูง	Low friction / Self-service	QR3 Manual Intervention Rate: เหตุการณ์จองที่ต้องให้ staff แก้ $\leq 1\%$ ของรายการจอง
“เครียด ดูไม่เป็นมืออาชีพ”	ความไม่แน่นอนสูง	Predictability / ลดความเสี่ยง	QR4 Booking Confidence Indicator Accuracy: การคาดการณ์ความพร้อมใช้งาน ณ เวลาเริ่มจองผิดพลาด $\leq 1\%$ (นิยามด้านล่าง)
“สุดท้ายไม่กล้าจองล่วงหน้า”	ผู้ใช้หลีกเลี่ยงฟีเจอร์หลัก	System trust	QR5 Feature Adoption (Advance Booking): อัตราการจองล่วงหน้าเพิ่ม $\geq 30\%$ หลังใช้งานต้นแบบ (ช่วง pilot)

4.3 นิยาม QR แบบ “มาตรฐานวิศวกรรม” (ให้ทดสอบได้จริง)

ด้านล่างนี้คือการ “เขียน QR ให้ครบ 6 ช่อง” แบบที่นักศึกษาควรวัดเป็น template

QR1: Booking Fulfillment Rate (อัตราการจองที่ใช้งานได้จริง)

1) นิยาม Metric : Booking Fulfillment Rate (BFR) = จำนวน booking ที่ผู้ใช้ “check-in ได้สำเร็จ” ÷ จำนวน booking ที่เริ่มเวลาแล้ว

2) Unit เปอร์เซนต์ (%)

3) Target $\geq 99\%$

4) Test Conditions (เงื่อนไขทดสอบ)

- ช่วงเวลาใช้งานจริง 7 วัน (pilot)
- รวมทุกประเภท booking: ห้องประชุม/โต๊ะ/อุปกรณ์
- รวมเหตุการณ์ walk-in และการเปลี่ยนแปลงหน่วยงาน

5) Measurement Method (วัดอย่างไร)

- เก็บ log ของ booking_id ที่เข้าสู่ “เวลาเริ่มต้น”
- ตรวจสอบสถานะว่ามี check-in สำเร็จหรือเกิด incident ประเภท “resource unavailable / conflict”
- รายงานสรุปเป็น %

6) Pass/Fail

- Pass ถ้า $BFR \geq 99\%$
- Fail ถ้า $< 99\%$ หรือมี incident รุนแรงที่ทำให้ “booking ใช้ไม่ได้” เกินเกณฑ์

QR1 สะท้อน “Trust” แบบจับต้องได้ที่สุด: ถ้าจองแล้วใช้ได้จริง = ระบบน่าเชื่อถือ

QR2: Status Update Latency (ความหน่วงการอัปเดตสถานะทรัพยากร)

1) นิยาม Metric : Status Update Latency (SUL) = เวลา (timestamp) ที่เกิดเหตุการณ์จริง - เวลา (timestamp) ที่ระบบสะท้อนสถานะใหม่

เหตุการณ์จริง เช่น:

- staff ทำ check-in / check-out
- ยกเลิกหน่วยงาน
- ปลดล็อกห้องหลัง no-show
- เปลี่ยนสถานะเป็น maintenance (ตามโจทย์มีสถานะ maintenance ของ resource)

2) Unit วินาที (sec)

3) Target ≤ 5 วินาที (95th percentile) (แนะนำให้ percentile เพราะค่าเฉลี่ยอาจหลอกได้)

4) Test Conditions

- จำลองโหลดพร้อมกัน (concurrent updates) 20 เหตุการณ์/นาที/สาขา
- วัดทั้ง peak hour และ off-peak

5) Measurement Method

- บันทึก event timestamp ตอน “กดปุ่ม” (client-side) และตอน “DB commit” (server-side)
- คำนวณ SUL แล้วสรุป p95

6) Pass/Fail

- Pass ถ้า $p95 \leq 5 \text{ sec}$
- Fail ถ้า $p95 > 5 \text{ sec}$ หรือมี outlier $> 30 \text{ sec}$ เกิน 1% (กำหนดเพื่อกันหลุดหนัก)

QR3: Manual Intervention Rate (สัดส่วนเหตุการณ์ที่ต้องให้ staff แก้)

1) นิยาม Metric : Manual Intervention Rate (MIR) = จำนวน booking ที่ต้องเปิดเคสให้ staff/admin override ÷ จำนวน booking ทั้งหมด

ตัวอย่าง “ต้องแก้”:

- conflict แล้วต้องย้ายห้องให้
- refund/ยกเลิกค่าปรับเฉพาะเคส
- ปลดล็อก/แก้สถานะด้วยมือ

(โจทย์มีส่วน penalty/reward และการอนุมัติจาก admin ซึ่งมักสร้าง manual workload)

2) Unit เปอร์เซ็นต์ (%)

3) Target $\leq 1\%$

4) Test Conditions

- ช่วง pilot 7 วัน
- เก็บทั้งเคสที่เกิดจาก walk-in/no-show/ระบบผิดพลาด

5) Measurement Method

- ทุกเคสที่ staff กด “override / resolve manually” ต้องสร้าง ticket_type ในระบบ
- นับจำนวน ticket_type ต่อ booking

6) Pass/Fail

- Pass ถ้า $MIR \leq 1\%$
- Fail ถ้าเกิน 1% หรือมี ticket บางประเภทพุ่งสูง (ต้องแยกดู root cause)

QR4: Booking Confidence Indicator Accuracy (ความแม่นยำของ “สัญญาณความมั่นใจ”)

เป็นตัวอย่าง “แปลงความรู้สึกเครียด/ไม่แน่นอน” ให้เป็น QR ที่วัดได้

แนวคิด: ระบบต้องบอกผู้ใช้ได้ว่า “จองนี้เสี่ยงแค่ไหน” แล้วความเสี่ยงนั้นต้องแม่นยำ

1) นิยาม Metric : ระบบแสดง Confidence Level (High/Medium/Low) ก่อนผู้ใช้กดยืนยันการจอง

จากข้อมูลเช่น: ความถี่ no-show, ความห่วงกังวล, walk-in rate, maintenance frequency ฯลฯ

Accuracy = สัดส่วนที่ระบบ “ทำนายถูก” ว่า booking นั้นจะสำเร็จ/มีปัญหา

2) Unit เปอร์เซนต์ (%)

3) Target ความผิดพลาด $\leq 1\%$ (เฉพาะกลุ่มที่ระบบขึ้นว่า “High confidence” ต้องพลาดน้อยมาก)

4) Test Conditions

- ใช้ข้อมูล pilot 7 วัน
- ตัวอย่างน้อย 200 booking (หรือกำหนดขั้นต่ำตามจำนวนจริง)

5) Measurement Method

- เก็บ predicted_confidence ก่อนยืนยันการจอง
- ตรวจสอบจริงว่ามี incident หรือไม่
- ทำ confusion matrix (ทำนายว่า “มั่นใจสูง” แต่ล้มเหลว = critical error)

6) Pass/Fail

- Pass ถ้า High-confidence bookings fail $\leq 1\%$
- Fail ถ้าเกินเกณฑ์ เพราะนี้ทำลาย trust มากที่สุด

หมายเหตุ: QR4 จะกลายเป็นงานเชิงวิศวกรรมข้อมูล/สถิติเล็ก ๆ แต่ยังอยู่ในขอบเขต Capstone ได้ดี

QR5: Feature Adoption (Advance Booking Adoption)

1) นิยาม Metric : Advance Booking Adoption Rate = จำนวน booking ที่ทำล่วงหน้า ≥ 1 ชม. $\div$ booking ทั้งหมด

2) Unit %

3) Target เพิ่มขึ้น $\geq 30\%$ เทียบ baseline ก่อนใช้ต้นแบบ

4-6) เงื่อนไข/วิธีวัด/ผ่านไม่ผ่าน ทำ A/B หรือ before-after แบบ pilot

คำเตือน: QR นี้ “ดีในเชิงธุรกิจ” แต่ควรเป็นรอง QRs เชิงระบบ (QR1-QR3) เพราะปัจจัยอื่นมีผล

4.4 เทคนิคสำคัญที่ทำให้นักศึกษา “ไม่หลุด” ตอนเขียน QR

เทคนิค A: QR ต้อง “ควบคุมได้” และ “วัดได้”

ถ้า QR วัดไม่ได้ → ถือว่ายังไม่ใช่ requirement

ตัวอย่างสิ่งที่ไม่ใช่ QR

- “ระบบต้องใช้ง่าย” (ยังไม่วัด)
- “ต้องปลอดภัยมาก” (ไม่กำหนด metric)
- “ต้องทันสมัย” (ไม่ใช่ requirement)

แปลงให้เป็น QR

- “Usability SUS score ≥ 80 ”
- “Login success rate $\geq 99.5\%$ ”
- “p95 response time ≤ 500 ms”

เทคนิค B: แยก “Target” กับ “Threshold”

- Target: ค่าที่อยากได้ที่สุด
- Threshold: ต่ำกว่านี้ถือว่า fail แน่แน่นอน

เช่น QR2:

- Target p95 ≤ 3 sec
 - Threshold p95 ≤ 5 sec
- (เขียนแบบนี้จะช่วยตอนเจรจากับ stakeholder และตอนทดสอบ)

เทคนิค C: อย่าผูก QR กับ “วิธีทำ”

QR คือ “ผลลัพธ์ที่ลูกค้าต้องสัมผัสได้” ไม่ใช่ “ใช้ Redis / ใช้ WebSocket / ใช้ AI”

4.5 Deliverables ที่ต้องส่งในรายงาน (รูปแบบที่แนะนำให้ศ.ทำตาม)

Deliverable 4.1: VOC Translation Matrix (เต็ม)

ตาราง 4 คอลัมน์ตามที่ทำด้านบน (Raw → Issue → Underlying → QR)

Deliverable 4.2: QR Specification Sheet (1 หน้า/1 QR)

สำหรับ QR1–QR4 ให้ทำเป็นชีตแบบเดียวกัน:

- Metric definition (สูตร)
- Unit
- Target/Threshold
- Test conditions
- Measurement method

- Pass/Fail criteria
- Risk ถ้าวัดไม่ได้ (เช่น ข้อมูลไม่พอ)

Deliverable 4.3: Traceability (ลิงก์จาก VoC → QR)

แสดงให้เห็นว่า QR แต่ละข้อ “มาจากคำพูด/ความต้องการไหน” (จะช่วยนักศึกษา เวลาถูกถามว่า “ทำไมตั้งตัวเลขนี้”)

ขั้นตอนที่ 5: แปลง QR/CTQ → ECs และสร้าง QFD

ในขั้นตอนที่ 4 เรามี QR/CTQ เป็น “ตัวเลขที่ลูกค้าต้องสัมผัสได้” (เช่น Booking Fulfillment $\geq 99\%$)

แต่ในขั้นตอนที่ 5 เราต้องตอบคำถามวิศวกรรมว่า:

เราต้อง “ปรับปรุงทางเทคนิคอะไร” (ECs) ถึงจะทำให้ QR บรรลุ?

ตามหลัก :

- QR = สิ่งที่ถูกความรู้สึก/สัมผัส
- EC = สิ่งที่มีออกมาแบบควบคุมได้และวัดได้ (Design for Measurement)

ใช้ Functional Decomposition แดกงาน/โมดูลที่จะทำให้ QR สำเร็จ

5.1 ตั้ง “Customer Requirements (CRs)” ให้เป็นชุดเดียว

5.1.1 สรุป CRs ที่จะใช้ (จาก VoC + เป้าหมายโจทย์)

CR1 — ลดการจองซ้ำซ้อน/จองแล้วใช้ไม่ได้

- อิงเป้าหมายโจทย์: ลด booking conflict $< 1\%$

CR2 — สถานะทรัพยากรถูกต้องและทันเวลา (Real-time)

- รองรับสถานะการจองหลายสถานะ (Pending/Confirmed/Checked-in/No-show...)

CR3 — ลด No-show ด้วยกลไกอัตโนมัติ (Auto penalty)

- อิงโจทย์: ลด no-show จาก 15% เหลือ $\leq 5\%$ โดยระบบค่าปรับอัตโนมัติ
- และกฎ no-show/penalty ที่ชัดเจน (เช่น ไม่ check-in ภายใน 30 นาที = no-show)

CR4 — การชำระเงิน/ออกบิลถูกต้องและไม่สะดุด

- เพราะระบบครอบคลุม Billing & Payment และช่องทางหลายแบบ (cash/card/QR/auto-debit/transfer)

CR5 — การแจ้งเตือนต้องทันเวลาและถูกคนถูกช่องทาง

- เพราะมี requirement แจ้งเตือนหลายเหตุการณ์และกำหนดเวลา “ทันที/ก่อน 24 ชม./1 ชม./หลัง no-show 30 นาที”

5.2 ใส่ “น้ำหนักความสำคัญ (Importance Weight)” ให้แต่ละ CR

ต้องมีน้ำหนัก 1–5 (หรือ 1–10) เพื่อสะท้อนว่า “เสี่ยงลูกค้า/โจทย์ให้ความสำคัญแค่ไหน”

5.2.1 วิธีตั้งน้ำหนักแบบ ไม่มั่ว

ให้ใช้ 3 แหล่งร่วมกัน:

1. VoC: เจ็บมาก/ปวดช้า/มีอาการร่วมสูง → น้ำหนักสูง
2. เป้าหมายโจทย์ที่ระบุตัวเลขชัด → น้ำหนักสูง
3. ถ้าพังแล้วกระทบธุรกิจ/ชื่อเสียง → น้ำหนักสูง

5.2.2 กำหนดน้ำหนักตัวอย่าง (1–5)

- CR1 ลดจองซ้ำซ้อน/จองแล้วใช้ไม่ได้ → 5 (เป็นเป้าหมายโจทย์ + core pain)
- CR2 Real-time status → 5 (รากของ trust/ความถูกต้อง)
- CR3 ลด no-show ด้วย auto penalty → 5 (เป็นเป้าหมายโจทย์ชัด)
- CR4 Billing/Payment ไม่สะดวก → 4 (กระทบรายได้/ข้อร้องเรียน, ช่องทางเยอะ)
- R5 Notification ถูกเวลา → 3 (สำคัญ แต่เป็นตัวเสริมให้ CR1–CR3 สำเร็จ)

5.3 สร้าง “Engineering Characteristics (ECs)” ที่ควบคุมและวัดได้

EC ต้อง “Control + Measure” ได้ ดังนั้น EC จะต้องเป็น “ตัวชี้วัดเชิงเทคนิค” ไม่ใช่ข้อพึงเจอร้อย ๆ

5.3.1 Functional Decomposition (แตกงานก่อนตั้ง EC)

แตกตามระบบในโจทย์: booking/resource/billing/penalty/notification

แล้วถามว่า “ส่วนไหนของระบบเป็นตัวทำให้ CR บรรลุ?”

ตัวอย่างแตกเชิงฟังก์ชัน

- Booking ต้องมี: กันชนกัน (concurrency), ตรวจสอบความพร้อม, lock/hold, เปลี่ยนสถานะ
- Resource status ต้องมี: event update pipeline, consistency rule, staff override ที่ถูก log
- No-show ต้องมี: detection timer 30 นาที + auto penalty + suspend booking ตามกฎ
- Payment ต้องมี: transaction success + reconciliation + fallback ช่องทาง (เช่น cash ชั่วคราว)
- Notification ต้องมี: delivery latency + delivery success + correct recipient/time window

5.3.2 ชุด ECs ตัวอย่าง

ตั้ง EC ให้ “วัดได้จริง” และโยงตรงกับ CR:

EC1 — Booking Conflict Rate (BCR)

- % ของ booking ที่เกิด conflict/overlap ในระบบ (ต่ำลง = ดี)

EC2 — Concurrency Control Correctness (CCC)

- อัตราความถูกต้องของกลไก lock/hold/transaction ในกรณี concurrent booking

EC3 — Resource Status Update Latency (SUL)

- เวลาที่สถานะทรัพยากรถูกอัปเดตหลังเหตุการณ์จริง (วินาที)

EC4 — No-show Detection + Penalty Execution Latency (NPL)

- เวลาตั้งแต่ “ครบ 30 นาทีไม่ check-in” → ระบบ mark no-show + คิดค่าปรับ + แจ้งเตือน/ระงับ ตามกฎ

EC5 — Payment Transaction Success Rate (PTSR)

- % ของ payment attempts ที่สำเร็จ (แยกตามช่องทางได้)

EC6 — Billing Calculation Accuracy (BCA)

- % ความถูกต้องของยอดสุทธิ (workspace+room+resource+penalty) เทียบกับสูตรใน requirement

EC7 — Notification Delivery Latency & Success (NDL/NSR)

- latency “ทันที/ก่อน 24 ชม./1 ชม./หลัง no-show 30 นาที” และอัตราส่งสำเร็จ

หมายเหตุ: EC2 กับ EC1 คล้ายกันแต่ไม่ซ้ำ

- EC2 = “กลไกทางเทคนิค” (ถูก/ไม่ถูกภายใต้ concurrency)
- EC1 = “ผลลัพธ์เชิงระบบ” (conflict ที่เกิดจริงลดลง)

5.4 สร้าง House of Quality (HoQ) แบบเต็มขั้น

HoQ มี 4 ส่วนหลัก:

1. แถวซ้าย: CRs + น้ำหนัก
2. คอลัมน์บน: ECs
3. กลางตาราง: ความสัมพันธ์ CR↔EC (strong/medium/weak)
4. ท้ายตาราง: คะแนนความสำคัญของ EC (Technical Importance)

5.4.1 นิยามสเกลความสัมพันธ์ (มาตรฐานที่แนะนำ)

ใช้ตัวเลขแบบ QFD คลาสสิก:

- Strong = 9 Medium = 3 Weak = 1 None = 0

5.4.2 HoQ Matrix (ตัวอย่าง “กรอกจริง”)

CR (น้ำหนัก) \ EC	EC1 BCR	EC2 CCC	EC3 SUL	EC4 NPL	EC5 PTSR	EC6 BCA	EC7 NDL/NSR
CR1 ลด booking conflict (<1%) (5)	9	9	3	1	0	0	1
CR2 Real-time status ถูกต้อง (5)	3	3	9	1	0	0	3
CR3 ลด no-show ≤5% ด้วย auto penalty (5)	1	1	3	9	0	3	9
CR4 Billing/Payment ไม่สะดุด (4)	0	0	0	3	9	9	1
CR5 Notification ถูกเวลา (3)	0	0	1	3	0	0	9

5.4.3 วิธีคิด Technical Importance (คำนวณให้ดูครบทุกคอลัมน์)

สูตร: $\text{Importance(ECj)} = \sum [\text{Weight(CRi)} \times \text{Relationship(CRi, ECj)}]$

คำนวณตัวอย่าง (ผมคิดให้ครบเพื่อเป็น template):

- $\text{EC1} = 5 \times 9 + 5 \times 3 + 5 \times 1 + 4 \times 0 + 3 \times 0 = 45 + 15 + 5 = 65$
- $\text{EC2} = 5 \times 9 + 5 \times 3 + 5 \times 1 + 4 \times 0 + 3 \times 0 = 65$
- $\text{EC3} = 5 \times 3 + 5 \times 9 + 5 \times 3 + 4 \times 0 + 3 \times 1 = 15 + 45 + 15 + 3 = 78$
- $\text{EC4} = 5 \times 1 + 5 \times 1 + 5 \times 9 + 4 \times 3 + 3 \times 3 = 5 + 5 + 45 + 12 + 9 = 76$
- $\text{EC5} = 5 \times 0 + 5 \times 0 + 5 \times 0 + 4 \times 9 + 3 \times 0 = 36$
- $\text{EC6} = 5 \times 0 + 5 \times 0 + 5 \times 3 + 4 \times 9 + 3 \times 0 = 15 + 36 = 51$
- $\text{EC7} = 5 \times 1 + 5 \times 3 + 5 \times 9 + 4 \times 1 + 3 \times 9 = 5 + 15 + 45 + 4 + 27 = 96$

5.4.4 สรุป “ลำดับความสำคัญ EC” (เอาไปใช้เลือกแนวคิด)

เรียงจากมากไปน้อย:

1. EC7 Notification (96)
2. EC3 Status Update Latency (78)
3. EC4 No-show & Penalty Latency (76)
4. EC1/EC2 Booking conflict + concurrency correctness (65)
5. EC6 Billing accuracy (51)
6. EC5 Payment success (36)

ประเด็น: ผลลัพธ์แบบนี้ “ไม่ผิด” แม้ Notification ดูขึ้นอันดับ 1 เพราะในโจทย์ notification ผูกกับ no-show/penalty และเวลาส่งแบบเข้มงวด (ทันที/24ชม./1ชม./หลัง no-show 30 นาที) แต่ในการทำจริง ทีมอาจ “normalize” หรือทบทวน weights หากเห็นว่า EC7 สูงเกินตรรกะธุรกิจ (นี้คือบทสนทนากับ stakeholder ที่ดี)

5.5 Roof of HoQ (ความสัมพันธ์ระหว่าง ECs)

Roof คือการบอกว่า “EC บางตัวดีขึ้นจะทำให้อีกตัวดีขึ้น/แย่ลง” (trade-off)

ตัวอย่างความสัมพันธ์สำคัญในเคสนี้:

- EC3 (Update latency) ดีขึ้น มักทำให้ EC1 (Conflict rate) ดีขึ้น เพราะข้อมูลสด ลดจองทับซ้อน (Positive correlation)
- EC2 (Lock/hold เข้มขึ้น) อาจทำให้ UX ช้าลง/transaction complexity เพิ่มขึ้น (อาจกระทบ latency บางจุด) → trade-off
- EC7 (notification เข้มและถี่) อาจเพิ่มโหลดระบบ ส่งผลต่อ EC3/EC4 ถ้า architecture ไม่ดี

ในรายงานให้นักศึกษาใส่ Roof แบบสัญลักษณ์ง่าย ๆ: ++ / + / - / -- พร้อมคำอธิบาย 1 บรรทัดต่อคู่

5.6 ตั้ง “ทิศทางการปรับปรุง (Direction)” + Target ของ EC (ให้วัดได้จริง)

EC ทุกตัวต้องระบุว่า “มากขึ้นดี” หรือ “น้อยลงดี” และมี target/threshold

ตัวอย่าง :

- EC1 Booking Conflict Rate: ↓ เป้าหมาย < 1% (ตรงเป้าโจทย์)
- EC3 Status Update Latency p95: ↓ เช่น $p95 \leq 5$ วินาที (คุณกำหนดตามความเหมาะสมระบบ)
- EC4 No-show → Penalty Latency p95: ↓ ต้องรองรับ “หลังเวลานัด 30 นาที” ใน requirement แจ้ง no-show และนิยาม no-show 30 นาที
- EC5 Payment Success Rate: ↑ เช่น $\geq 99\%$ (แยกตามช่องทาง)
- EC6 Billing Accuracy: ↑ เช่น 100% ใน test cases ตามสูตรค่าใช้จ่าย 4 องค์ประกอบ
- EC7 Notification Success: ↑ และ latency ต้องตรงตารางเวลาแจ้งเตือน

เคล็ดลับ: ถ้าไม่รู้จะตั้งตัวเลขเท่าไร ให้ทำ “baseline measurement” จาก mock/prototype แล้วค่อยตั้ง target แบบมีเหตุผล (จะทำในขั้น Verification Plan)

5.7 Requirements Traceability (โยงกลับเพื่อกันโดนถาม)

สิ่งที่กรรมการชอบถามคือ “ทำไม EC นี้ต้องทำ?” คำตอบที่ดีที่สุดคือ “โยงจาก CR/QR” แบบ trace ตัวอย่าง trace ที่เขียนในรายงาน:

- เป้า “ลด booking conflict <1%” → CR1 → EC1 (BCR), EC2 (CCC), EC3 (SUL)
- เป้า “ลด no-show เหลือ ≤5%” + กฎ no-show/penalty → CR3 → EC4 (NPL), EC7 (NDL/NSR), EC6 (BCA)
- ตารางแจ้งเตือนหลายเหตุการณ์และเวลาชัด → CR5 → EC7 เป็นแกนหลัก

Output ของขั้นตอนนี้

1. รายการ CR + **น้ำหนัก** (พร้อมเหตุผลสั้น ๆ)
2. รายการ EC ที่ควบคุมและวัดได้ (พร้อมนิยาม + หน่วย + ทิศทาง)
3. ตาราง HoQ (relationship 9/3/1) + การคำนวณ **Technical Importance**
4. Roof (trade-off) อย่างน้อย 3–5 คู่ที่สำคัญ
5. Traceability (CR→EC) 1 หน้า

ขั้นตอนที่ 6 การสร้างแนวคิดและการคัดเลือกแนวคิด (Concept Generation & Concept Selection)

วัตถุประสงค์ของขั้นตอนนี้

ขั้นตอนนี้มีหน้าที่ตอบคำถามสำคัญ 3 ข้อ:

1.  เรามี “วิธีแก้ปัญหา” ได้กี่แนวทาง (ไม่ด่วนตัดสิน)
2.  แต่ละแนวทางตอบโจทย์ทางวิศวกรรม (ECs) ดีแค่ไหน
3.  แนวคิดใด “เหมาะสมที่สุด” ภายใต้ข้อจำกัดจริงของ Capstone

! จุดสำคัญ การ “เลือกแนวคิด” ต้อง **ไม่ใช่ความรู้สึก** แต่ใช้ Engineering Characteristics (ECs) + น้ำหนักจาก HoQ เป็นเกณฑ์

6.1 การสร้างแนวคิด (Concept Generation)

6.1.1 หลักการสำคัญ (สิ่งที่นักศึกษาต้องเข้าใจ)

- แนวคิด = สถาปัตยกรรมหรือแนวทางแก้ปัญหาโดยรวม
- ยังไม่ใช่ implementation detail (ยังไม่เลือก framework / ภาษา)
- แต่ต้อง ทำนายประสิทธิภาพเชิงตัวเลขได้คร่าว ๆ

แนวคิดที่ดีต้อง:

- ตอบ Root Cause (ไม่ใช่แค่อาการ)
- เชื่อมกับ ECs ที่สำคัญจาก HoQ
- มีข้อดี-ข้อจำกัดชัดเจน

6.1.2 ทบทวน ECs ที่ “สำคัญสูง” (จาก HoQ)

จากขั้นตอนที่ 5 เราได้ลำดับความสำคัญ EC ดังนี้:

1. EC7 – Notification Delivery Latency & Success
2. EC3 – Resource Status Update Latency (p95)
3. EC4 – No-show Detection & Penalty Latency
4. EC1/EC2 – Booking Conflict Rate & Concurrency Correctness
5. EC6 – Billing Accuracy
6. EC5 – Payment Success Rate

ดังนั้น แนวคิดที่ “ดี” ต้องเด่นอย่างน้อยใน EC3, EC4, EC1/2

ถ้าแนวคิดใดทำ EC เหล่านี้แย่ → ตกรอบได้เลย

6.1.3 แนวคิดที่ 1: Strict Reservation Lock Architecture (SRL)

แนวคิดโดยรวม

“จองแล้ว = ล็อกจริง ไม่มีใครแย่งได้”

สถาปัตยกรรมเชิงแนวคิด

- ทุกการจองต้องผ่าน central reservation service
- ใช้ strict locking / transaction-based booking
- Walk-in ต้อง “ขอสิทธิ์” จากระบบเดียวกับ booking
- ถ้า resource ถูกจอง → walk-in ถูกปฏิเสธอัตโนมัติ

การตอบ ECs (เชิงคาดการณ์)

- EC1 Booking Conflict Rate: ต่ำมาก (<0.5%)
- EC2 Concurrency Correctness: สูงมาก
- EC3 Status Update Latency: ปานกลาง (มี lock)

- EC4 No-show & Penalty: ทำได้ดี
- EC7 Notification: ดี (event-driven)

ข้อดี

- ความถูกต้องสูง
- Trust ของผู้ใช้สูงมาก
- เหมาะกับ Premium Member

ข้อจำกัด

- ระบบแข็ง (Rigid)
- อาจสร้าง friction กับ walk-in
- Latency อาจเพิ่มเมื่อมี concurrent สูง

6.1.4 แนวคิดที่ 2: Hybrid Reservation with Walk-in Buffer (HRW)

แนวคิดโดยรวม

“ยอมกันพื้นที่บางส่วนให้ walk-in อย่างควบคุมได้”

สถาปัตยกรรมเชิงแนวคิด

- แบ่ง resource เป็น 2 pool:
 - Reserved Pool
 - Walk-in Buffer Pool
- ระบบใช้ rule-based allocation
- มี dynamic rebalancing (เช่น ปล่อย buffer เมื่อใกล้เวลา)

การตอบ ECs (เชิงคาดการณ์)

- EC1 Booking Conflict Rate: ต่ำ-ปานกลาง ($\approx 1\%$)
- EC2 Concurrency: ดี
- EC3 Status Latency: ดี (ไม่ล็อกแข็ง)
- EC4 No-show: ดี
- EC7 Notification: ดี

ข้อดี

- ยืดหยุ่นสูง
- รองรับพฤติกรรมจริงของ coworking
- UX ดีทั้ง member และ staff

ข้อจำกัด

- Logic ซับซ้อน
- ต้องออกแบบ rule ให้ดี ไม่งั้น conflict กลับมา

6.1.5 แนวคิดที่ 3: Prediction-based Reservation Control (PRC)

แนวคิดโดยรวม

“ใช้ข้อมูลอดีตทำนายความเสี่ยงก่อนจอง”

สถาปัตยกรรมเชิงแนวคิด

- ระบบประเมิน Risk Score ต่อ booking
- ใช้ข้อมูล:
 - no-show rate
 - walk-in frequency
 - time-of-day
- จองที่ risk สูง → ระบบเตือนหรือจำกัดสิทธิ์

การตอบ ECs (เชิงคาดการณ์)

- EC1 Conflict Rate: ต่ำ (ถ้าทำนายแม่น)
- EC3 Latency: ดี
- EC4 No-show: ดีมาก
- EC7 Notification: ดีมาก

ข้อดี

- ฉลาด
- UX ดี
- ลด no-show เชิงรุก

ข้อจำกัด

- ต้องมีข้อมูลมาก
- ความเสี่ยงด้านความแม่นยำ
- ซับซ้อนเกิน Capstone บางกรณี

6.2 การคัดเลือกแนวคิด (Concept Selection)

เครื่องมือหลัก: Weighted Decision Matrix (WDM)

เกณฑ์ = ECs จาก HoQ

น้ำหนัก = Technical Importance จาก HoQ

6.2.1 กำหนดเกณฑ์และน้ำหนัก (จาก HoQ)

EC	น้ำหนัก (Weight)
EC7 Notification	5
EC3 Status Latency	5

EC	น้ำหนัก (Weight)
EC4 No-show/Penalty	5
EC1 Booking Conflict	4
EC2 Concurrency	4
EC6 Billing Accuracy	3
EC5 Payment Success	2

6.2.2 ให้คะแนนแต่ละแนวคิด (1-5) 1 = แย่มาก, 3 = พอใช้, 5 = ดีเยี่ยม ต้องให้เหตุผลทุกคะแนน

Weighted Decision Matrix

EC	Weight	SRL	HRW	PRC
EC7 Notification	5	4	4	5
EC3 Status Latency	5	3	4	4
EC4 No-show/Penalty	5	4	4	5
EC1 Conflict Rate	4	5	4	4
EC2 Concurrency	4	5	4	3
EC6 Billing Accuracy	3	4	4	4
EC5 Payment Success	2	4	4	4

6.2.3 คำนวณคะแนนรวม (แสดงให้ดูครบ)

SRL

$$= 5 \times 4 + 5 \times 3 + 5 \times 4 + 4 \times 5 + 4 \times 5 + 3 \times 4 + 2 \times 4 = 20 + 15 + 20 + 20 + 20 + 12 + 8 = 115$$

HRW

$$= 5 \times 4 + 5 \times 4 + 5 \times 4 + 4 \times 4 + 4 \times 4 + 3 \times 4 + 2 \times 4 = 20 + 20 + 20 + 16 + 16 + 12 + 8 = 112$$

PRC

$$= 5 \times 5 + 5 \times 4 + 5 \times 5 + 4 \times 4 + 4 \times 3 + 3 \times 4 + 2 \times 4 = 25 + 20 + 25 + 16 + 12 + 12 + 8 = 118$$

6.3 ผลการตัดสินใจเลือกแนวคิด

แนวคิดที่ได้คะแนนสูงสุด:



Concept C – Prediction-based Reservation Control (PRC)

คะแนนรวม = 118

6.4 เหตุผลเชิงวิศวกรรมที่เลือก PRC

1. ตอบ EC ที่สำคัญสูงสุด (EC7, EC3, EC4) ได้ดีที่สุด
2. แก้ปัญหาเชิงรุก ไม่ใช่รอให้ conflict เกิด
3. ลด no-show ตามเป้าหมายได้ชัด



หมายเหตุสำหรับนักศึกษา

ถ้าข้อมูลไม่พอ → สามารถ “ลดระดับ” PRC ให้เป็น rule-based prediction
โดยยังรักษาแนวคิดไว้ (ไม่จำเป็นต้องเป็น AI เต็มรูปแบบ)

6.5 สิ่งที่ได้จากขั้นตอนนี้ (Deliverables)

1. แนวคิดอย่างน้อย 3 แนวทาง พร้อมคำอธิบาย
2. Weighted Decision Matrix (มีน้ำหนัก + เหตุผล)
3. แนวคิดที่ถูกเลือก + เหตุผลเชิงวิศวกรรม
4. หลักฐานว่า “การเลือกไม่ได้มาจากความรู้สึก”

ขั้นตอนที่ 7 Preliminary Design Review (PDR)

(Concept ที่เลือก: Prediction-based Reservation Control – PRC)

วัตถุประสงค์ของ PDR

PDR คือ “จุดตรวจทางวิศวกรรม” เพื่อยืนยันว่า:

1.  แนวคิดที่เลือก สามารถแปลงเป็นระบบจริงได้
2.  ทุก Quantifiable Requirement (QR/CTQ) มีองค์ประกอบระบบรองรับ
3.  ระบบถูกแตกเป็นส่วนย่อยที่ ออกแบบ-พัฒนา-ทดสอบได้
4.  ความเสี่ยงหลักถูกมองเห็น ก่อน ลงมือทำจริง



PDR ยังไม่ใช้การลงรายละเอียดโค้ด

แต่ต้องละเอียดพอที่ “ถ้าให้ทีมอื่นไปทำต่อ ต้องไม่หลง”

7.1 ภาพรวมแนวคิดที่ผ่านการคัดเลือก (Concept Recap)

แนวคิดที่เลือก Prediction-based Reservation Control (PRC)

แก่นแนวคิด

- ใช้ข้อมูลพฤติกรรมการใช้งานจริง (no-show, walk-in, time-of-day)
- ประเมิน “ความเสี่ยงของการจอง” ก่อนผู้ใช้ยืนยัน
- ควบคุมการจองเชิงรุก แทนการแก้ปัญหาภายหลัง
- ผสานกับกลไก auto no-show detection + penalty + notification

7.2 การกำหนด System Architecture ระดับสูง (High-Level Architecture)

7.2.1 ภาพรวมสถาปัตยกรรมเชิงระบบ

ระบบถูกออกแบบเป็น event-driven, service-oriented architecture เพื่อรองรับ:

- Real-time update
- การคำนวณความเสี่ยง
- การแจ้งเตือนหลายเหตุการณ์

Subsystem หลัก

1. User Interface (Web/Mobile)
2. Reservation & Resource Control Layer
3. Prediction & Rule Engine
4. Billing & Penalty Engine
5. Notification Service
6. Central Database & Event Log

7.3 การแตกองค์ประกอบระบบ (System Decomposition)

ตามหลัก PDR ต้องนิยาม Configuration Items (CIs) ให้ชัดเจน

7.3.1 Configuration Items (CIs)

CI-1: Reservation & Resource Management Engine

หน้าที่

- จัดการ booking lifecycle
- ตรวจสอบความพร้อมของทรัพยากร
- ป้องกัน booking conflict
- บังคับ state transition

QR ที่รองรับ

- QR1 Booking Fulfillment Rate $\geq 99\%$
- QR2 Status Update Latency ≤ 5 sec

CI-2: Prediction & Risk Assessment Module

หน้าที่

- ประเมิน Risk Score ก่อนยืนยันจอง
- ใช้ rule-based + historical statistics
- จัดระดับความมั่นใจ (High / Medium / Low)

QR ที่รองรับ

- QR4 Booking Confidence Indicator Accuracy
- สนับสนุน QR1 ทางอ้อม (ลด conflict เชิงรุก)

CI-3: No-show Detection & Penalty Engine

หน้าที่

- ตรวจจับ no-show (ไม่ check-in ภายใน 30 นาที)
- คิดค่าปรับอัตโนมัติ
- ระบุสิทธิ์การจองตาม policy

QR ที่รองรับ

- QR3 Manual Intervention Rate $\leq 1\%$
- QR ลด no-show $\leq 5\%$

CI-4: Billing & Payment Processing Module

หน้าที่

- คำนวณค่าใช้จ่ายรวม (workspace + room + resource + penalty)
- เชื่อม payment gateway
- ออกใบแจ้งหนี้/ใบเสร็จ

QR ที่รองรับ

- QR Billing Accuracy
- QR Payment Success Rate

CI-5: Notification & Communication Service

หน้าที่

- แจ้งเตือนหลายเหตุการณ์:
 - ยืนยันการจอง
 - เตือนล่วงหน้า
 - แจ้ง no-show / penalty
- รองรับหลายช่องทาง (email / in-app / LINE)

QR ที่รองรับ

- QR Notification Delivery Latency & Success

7.4 Interface Control Documents (ICDs)

7.4.1 ตัวอย่าง ICD: CI-1 ↔ CI-2

Interface

- REST API: /api/v1/reservations/assess-risk

Input

- user_id
- resource_id
- time_slot
- booking_history_summary

Output

- risk_score (0–1)
- confidence_level (High/Medium/Low)
- advisory_message

Performance Requirement

- Response time ≤ 300 ms (p95)

7.4.2 ICD: CI-3 ↔ CI-5

Trigger Event

- No-show detected (T + 30 นาที)

Payload

- booking_id
- penalty_amount
- suspension_status

Notification SLA

- Notification dispatched ≤ 60 วินาทีหลัง event

7.5 Requirements Traceability (PDR Version)

พิสูจน์ว่า “ไม่มี QR ไตตกหล่น”

QR / CTQ	CI ที่รองรับ	กลไก
QR1 Booking Fulfillment $\geq 99\%$	CI-1, CI-2	conflict control + risk gating
QR2 Status Latency ≤ 5 sec	CI-1, CI-5	event-driven update
QR3 Manual Intervention $\leq 1\%$	CI-3	auto no-show + penalty
QR4 Confidence Accuracy $\leq 1\%$	CI-2	prediction validation
QR5 Adoption Increase	CI-1, CI-2	trust & transparency

7.6 Verification Plan (ระดับ PDR)

ใน PDR ต้อง “บอกว่าจะพิสูจน์อย่างไร” แม้ยังไม่ทดสอบจริง

ตัวอย่าง Verification Method

QR1 Booking Fulfillment

- **Method:** Simulation + Pilot logging
- **Metric:** successful check-in / total booking
- **Pass:** $\geq 99\%$

QR2 Status Latency

- **Method:** Event timestamp comparison
- **Metric:** p95 latency
- **Pass:** ≤ 5 sec

QR3 No-show Handling

- **Method:** Forced no-show test cases
- **Metric:** auto penalty execution success
- **Pass:** $\geq 99\%$ cases no manual override

7.7 Preliminary Risk Analysis (Early Risk Register)

7.7.1 ความเสี่ยงด้านเทคนิค

Risk	ผลกระทบ	แนวทางลดความเสี่ยง
ข้อมูลไม่พอสำหรับ prediction	Risk score ไม่แม่นยำ	เริ่มด้วย rule-based
Latency สูงช่วง peak	QR2 fail	async processing + queue
Notification ล่าช้า	Trust ลด	retry + monitoring

7.7.2 ความเสี่ยงด้านการใช้งานจริง

Risk	ผลกระทบ	แนวทาง
Staff ไม่ใช้ระบบครบชั้น	data ไม่สะท้อนจริง	training + UI simplification
สมาชิกต่อต้าน penalty	adoption ลด	transparency + warning

7.8 Allocated Baseline (ผลลัพธ์หลักของ PDR)

เมื่อจบ PDR โครงการได้:

1.  สถาปัตยกรรมระบบที่ชัดเจน
2.  Configuration Items ที่นิยามครบ
3.  Interface ระหว่างระบบย่อย
4.  Verification strategy สำหรับทุก QR
5.  ความเสี่ยงหลักที่รับรู้และมีแผนรับมือ

นี่คือ **Allocated Baseline** ที่จะถูกใช้เป็นฐานในการ



Detailed Design & Implementation

สิ่งที่ยัง “ไม่ทำ” ใน PDR

-  ยังไม่เขียนโค้ด
-  ยังไม่เลือก framework เจาะจง
-  ยังไม่ optimize performance จริง

ขั้นตอนที่ 8 Detailed Design & Work Breakdown Structure (WBS)

(Concept: Prediction-based Reservation Control – PRC)

วัตถุประสงค์ของขั้นตอน WBS

WBS มีหน้าที่ “แปลงสถาปัตยกรรมจาก PDR → งานที่ทำจริงได้” โดยต้องตอบให้ได้ว่า:

1.  งานทั้งหมดของโครงการมีอะไรบ้าง
2.  งานใดเป็นงานวิศวกรรมหลัก (Engineering Work)
3.  งานใดรองรับการทดสอบ QR/CTQ
4.  งานใดเหมาะกับนักศึกษาที่คน / ทำลำดับใดก่อน-หลัง

! หลักการสำคัญ ทุก QR ต้องมีงานใน WBS รองรับ ถ้า QR ใด “ไม่มีงาน” → แสดงว่ายังทำไม่ครบ

8.1 หลักการสร้าง WBS

กฎ 3 ข้อที่ต้องย้ากับนักศึกษา

1. WBS ไม่ใช่ Timeline (ยังไม่ใส่วัน)
2. WBS ไม่ใช่ To-do list เล็ก ๆ
3. WBS ต้องแตกจนถึงระดับที่
 “รู้ว่าใครทำ / ส่งอะไร / ตรวจอะไร”

ระดับ WBS ที่เหมาะกับ Capstone

- Level 1: โครงการ
- Level 2: Subsystem / Phase
- Level 3: Work Package
- (Level 4 ทำได้ถ้าทีมใหญ่ แต่ไม่จำเป็น)

8.2 WBS Level 1–2 (ภาพรวมทั้งโครงการ)

Level 1: Coworking Space Management System (PRC)

Level 2: โครงสร้างหลัก 7 หมวด

1. Project Management & Documentation
2. System Architecture & Detailed Design
3. Reservation & Resource Management (CI-1)
4. Prediction & Risk Assessment (CI-2)
5. No-show & Penalty Engine (CI-3)
6. Billing, Payment & Notification (CI-4, CI-5)
7. Integration, Testing & Verification

หมวดเหล่านี้สอดคล้องกับ CI จาก PDR แบบ 1:1 → ง่ายต่อการ trace QR ↔ งาน ↔ ผลลัพธ์

8.3 WBS ระดับละเอียด (Level 3 – Work Packages)

ด้านล่างคือ WBS ตัวอย่างที่ “แตกจริง” พร้อมคำอธิบายว่า งานนี้ทำไปเพื่ออะไร

1. Project Management & Documentation

1.1 Project Planning & Control

- 1.1.1 จัดทำ Project Charter & Scope Baseline
- 1.1.2 จัดทำ Requirement Traceability Matrix (RTM)
- 1.1.3 การประชุมติดตามความคืบหน้า (Sprint review / Weekly review)

1.2 Engineering Documentation

- 1.2.1 System Requirement Specification (SRS)
- 1.2.2 Architecture Design Document (จาก PDR)
- 1.2.3 Verification & Validation Plan



เหตุผล:

ต้องเห็นว่าโครงการ “ควบคุมได้” ไม่ใช่เขียนโค้ดอย่างเดียว

2. System Architecture & Detailed Design

2.1 Data & State Modeling

- 2.1.1 ออกแบบ Resource State Model
(Available / Reserved / Checked-in / No-show / Maintenance)
- 2.1.2 Booking Lifecycle & State Transition Diagram

2.2 Interface Design (ICD)

- 2.2.1 API ระหว่าง Reservation ↔ Prediction
- 2.2.2 Event schema สำหรับ no-show / penalty / notification



รองรับ QR2, QR4 โดยตรง

3. Reservation & Resource Management (CI-1)

3.1 Core Booking Engine

- 3.1.1 Implement booking creation / modification / cancellation
- 3.1.2 Time-slot validation & availability check

3.2 Conflict Prevention & Concurrency Control

- 3.2.1 Resource locking / hold mechanism
- 3.2.2 Concurrent booking simulation & test

3.3 Check-in / Check-out Handling

- 3.3.1 Member check-in workflow
- 3.3.2 Staff override & logging

 ร่องรับ QR1 Booking Fulfillment, QR2 Status Latency

4. Prediction & Risk Assessment Module (CI-2)

4.1 Data Preparation & Feature Definition

- 4.1.1 Define no-show rate, walk-in rate, time-based factors
- 4.1.2 Aggregate historical booking statistics

4.2 Risk Scoring Logic

- 4.2.1 Rule-based risk scoring (baseline)
- 4.2.2 Confidence level classification (High/Medium/Low)

4.3 Explainability & UI Support

- 4.3.1 Generate advisory message (เหตุผลที่ risk สูง)
- 4.3.2 Display confidence indicator ก่อนยืนยันการจอง

 ร่องรับ QR4 (Confidence Accuracy)

5. No-show Detection & Penalty Engine (CI-3)

5.1 No-show Detection Logic

- 5.1.1 Timer-based detection (T + 30 นาที)
- 5.1.2 Status transition → No-show

5.2 Penalty & Policy Enforcement

- 5.2.1 Penalty calculation rules
- 5.2.2 Account suspension / restriction logic

5.3 Exception Handling

- 5.3.1 Admin override workflow
- 5.3.2 Audit log for penalties

 ร่องรับ QR3 ลด manual intervention, ลด no-show

6. Billing, Payment & Notification (CI-4, CI-5)

6.1 Billing Calculation

- 6.1.1 Workspace / Room / Resource fee calculation
- 6.1.2 Penalty integration into billing

6.2 Payment Processing

- 6.2.1 Payment gateway integration
- 6.2.2 Payment failure & retry handling

6.3 Notification System

- 6.3.1 Booking confirmation & reminder
- 6.3.2 No-show & penalty notification
- 6.3.3 Multi-channel delivery (email / in-app)

 ร่องรับ QR5 Adoption, EC7 Notification

7. Integration, Testing & Verification

7.1 System Integration

- 7.1.1 CI-to-CI integration testing
- 7.1.2 End-to-end booking scenarios

7.2 Verification of QR/CTQ

- 7.2.1 Test Booking Fulfillment Rate (QR1)
- 7.2.2 Measure Status Update Latency p95 (QR2)
- 7.2.3 Validate Risk Prediction Accuracy (QR4)

7.3 Pilot & Evaluation

- 7.3.1 Pilot deployment (simulated users)
- 7.3.2 Collect metrics & analyze results

 ขั้นนี้คือ “ตอบกรรมการได้เต็มปากว่า QR ผ่านหรือไม่”

8.4 Traceability: WBS ↔ QR (สรุปแบบสอน)

QR / CTQ	WBS หมวดที่รองรับ
----------	-------------------

QR1 Booking Fulfillment $\geq$ 99%	3.1, 3.2, 3.3, 7.2
------------------------------------	--------------------

QR2 Status Latency $\leq$ 5 sec	2.1, 3.2, 6.3
---------------------------------	---------------

QR3 Manual Intervention $\leq$ 1%	5.1, 5.2, 5.3
-----------------------------------	---------------

QR4 Confidence Accuracy $\leq$ 1%	4.1, 4.2, 4.3
-----------------------------------	---------------

QR5 Adoption Increase	4.3, 6.3, 7.3
-----------------------	---------------

 ตารางนี้ช่วยให้นักศึกษา “ไม่โดนถามจนไปไม่ถูก” ตอนสอบ

8.5 สิ่งที่ WBS ชูนี้บอก

1. Engineering Design $\neq$ Coding
2. ทุก requirement ต้องมีงานรองรับ
3. งานทดสอบสำคัญพอ ๆ กับงานพัฒนา
4. Capstone ที่ดีต้อง “วัดผลได้”

ขั้นตอนที่ 9 Project Scheduling & Milestone Planning

วัตถุประสงค์ของขั้นตอน Scheduling

ขั้นตอนนี้มีหน้าที่ตอบคำถาม 5 ข้ออย่างเป็นระบบ:

1.  โครงการจะใช้เวลาที่สัปดาห์
2.  งานใดต้องทำก่อน-หลัง (Dependency)
3.  จุดตรวจสำคัญ (Milestones) อยู่ตรงไหน
4.  ภาระงานกระจายเหมาะกับทีม Capstone หรือไม่
5.  งานใดอยู่บน Critical Path (เข้าไม่ได้)

! หลักคิด WBS บอกว่า “ต้องทำอะไร” Scheduling บอกว่า “ทำเมื่อไร และลำดับอย่างไร”

9.1 สมมติฐานพื้นฐานของตารางเวลา (Assumptions)

เพื่อให้เป็น เอกสารตัวอย่างที่นักศึกษาเอาไปใช้ได้จริง

เรากำหนดสมมติฐานดังนี้:

- ระยะเวลาโครงการ: 14 สัปดาห์
- ทีม: 3-4 คน
- ทำงานคู่ขนานได้บางส่วน
- มีการสอบ/ส่งงานอื่น $\rightarrow$ ต้องเผื่อ buffer

9.2 โครงสร้าง Phase ของโครงการ (Phase-based Scheduling)

แนะนำให้นักศึกษา “จัดโครงการเป็น Phase”

เพราะกรรมการเข้าใจง่าย และผูกกับ Milestone ได้ชัด

Phase หลัก 6 Phase

1. Project Initiation & Requirement Finalization
2. System Design (PDR Completion)
3. Detailed Design & Core Implementation
4. Feature Integration & Advanced Logic

5. Verification, Validation & Pilot
6. Finalization & Presentation

9.3 การแมป WBS → Phase → สัปดาห์

Phase 1: Project Initiation & Requirement (Week 1-2)

เป้าหมาย: ล็อก “โจทย์ + QR + ขอบเขต” ให้แน่นก่อนเขียนระบบ

WBS	งาน	ระยะเวลา
1.1	Project Charter / Scope Baseline	3 วัน
1.2	Requirement Traceability Matrix	2 วัน
2.1	Data & State Modeling (Draft)	3 วัน
2.2	ICD (Draft)	2 วัน



Milestone M1:



Requirements Baseline Approved

Phase 2: System Design (Week 3-4)

เป้าหมาย: สรุปสถาปัตยกรรมและพร้อม “ลงมือทำโดยไม่หลง”

WBS	งาน	ระยะเวลา
2.1	State Transition Final	3 วัน
2.2	ICD Final	3 วัน
7.1	Integration Strategy Planning	2 วัน
7.2	Verification Plan (QR-based)	2 วัน



Milestone M2 (PDR):



Preliminary Design Review Passed

Phase 3: Core Implementation (Week 5–7)

เป้าหมาย: สร้าง “แกนระบบ” ที่ทุกอย่างจะพึ่งพา

WBS	งาน	ระยะเวลา
3.1	Core Booking Engine	7 วัน
3.2	Conflict & Concurrency Control	5 วัน
3.3	Check-in / Check-out	3 วัน
6.1	Billing Core Logic	3 วัน

 งานใน Phase นี้อยู่บน **Critical Path** ถ้าช้า → ทุกอย่างช้า

 **Milestone M3:**

 *Core System Functional*

Phase 4: Advanced Logic & Intelligence (Week 8–10)

เป้าหมาย: ทำให้ระบบ “ฉลาด + อัตโนมัติ” ตาม Concept PRC

WBS	งาน	ระยะเวลา
4.1	Feature Definition & Data Prep	3 วัน
4.2	Risk Scoring Logic	4 วัน
5.1	No-show Detection	3 วัน
5.2	Penalty Enforcement	3 วัน
6.3	Notification Engine	4 วัน

 **Milestone M4:**

 *All Major Features Integrated*

Phase 5: Integration, Testing & Verification (Week 11–12)

เป้าหมาย: พิสูจน์ว่า QR/CTQ ผ่านจริง ไม่ใช่แค่ “คิดว่าผ่าน”

WBS	งาน	ระยะเวลา
7.1	End-to-End Integration Test	4 วัน
7.2	QR Verification Tests	5 วัน
7.3	Pilot & Metrics Collection	3 วัน

 **Milestone M5:**  *QR Verification Completed*

Phase 6: Finalization & Presentation (Week 13–14)

เป้าหมาย: เตรียมสอบ/นำเสนออย่างมั่นใจ

WBS	งาน	ระยะเวลา
1.2	Final Documentation	5 วัน
7.3	Results Analysis	3 วัน
–	Final Presentation & Demo	3 วัน



Milestone M6 (Final):



Capstone Project Completed

9.4 Gantt Chart (อธิบายเชิงแนวคิด)

ในรายงานให้นักศึกษาวาด Gantt Chart จากข้อมูลนี้ (ไม่จำเป็นต้องซับซ้อน แค่เห็น Phase + overlap)

ตัวอย่าง Overlap ที่ “ทำได้จริง”

- Phase 3 (Core Implementation) ↔ Phase 4 (เริ่มเตรียม Risk Logic ได้)
- Documentation ทำ คู่ขนานตลอด ไม่ใช่ท้ายเทอม

9.5 Critical Path Analysis (สอนให้คิดแบบวิศวกร)

งานบน Critical Path (ตัวอย่าง)

- 3.1 Core Booking Engine
- 3.2 Conflict Control
- 4.2 Risk Scoring Logic
- 7.2 QR Verification

ถ้างานเหล่านี้นี้ช้า = โครงการช้า → ควรจัดคนเก่ง/เวลามากกว่างานอื่น

9.6 Buffer & Risk-aware Scheduling

แนะนำ Buffer ที่ “ควรมี”

- Technical Buffer: 10–15% ของ Phase 3–4
- Academic Buffer: เพื่อสอบ/กิจกรรมอื่น 1 สัปดาห์

ตัวอย่างการใส่ Buffer

- ตั้ง Milestone M4 ให้จบสัปดาห์ 10
- แต่จริง ๆ plan ให้เสร็จปลายสัปดาห์ 9

9.7 สิ่งที่กรรมการดูจาก Scheduling

1. งานเรียงลำดับสมเหตุสมผลไหม
2. มี Milestone ที่ตรวจได้จริงหรือไม่
3. มีการเผื่อความเสี่ยงหรือไม่
4. QR ถูกทดสอบ “ก่อนสอบ” หรือเปล่า

! ถ้า QR ยังไม่ถูกทดสอบ แต่ขึ้นสอบ = Capstone ยังไม่สมบูรณ์

Output ของขั้นตอน Scheduling

1. ตาราง Phase + Week
2. รายการ Milestones (M1–M6)
3. Gantt Chart
4. Critical Path Identification
5. Buffer Strategy

ขั้นตอนที่ 10 Verification & Validation (V&V) + Final Evaluation

10.1 ความต่างที่นักศึกษาต้องเข้าใจ (ก่อนเริ่ม)

Verification vs Validation

-  **Verification:** “เราสร้างระบบ ถูกต้องตาม *requirement* ไหม?”
→ วัดด้วย QR/CTQ, ทดสอบเชิงเทคนิค, Pass/Fail ชัดเจน
-  **Validation:** “สิ่งที่เราสร้าง ใช้สิ่งที่ลูกค้าต้องการจริง ไหม?”
→ วัดด้วย VoC/การใช้งานจริง/ความพึงพอใจ/พฤติกรรม

จำง่าย ๆ: Verification = ถูกสเปก, Validation = ถูกใจลูกค้า

10.2 V&V Strategy (แผนภาพรวม)

เพื่อให้เป็นวิศวกรรม “ครบ” เราจะใช้ 4 ขั้นตอนทดสอบ:

1. Unit Test (แต่ละโมดูลย่อย)
2. Integration Test (CI-to-CI)
3. System Test (End-to-End)
4. Pilot / User Validation (ทดสอบกับผู้ใช้ตัวแทน)

10.3 นิยาม QR ที่จะทดสอบ (Baseline)

จากชั้น 4-5 เรามี QR หลัก (ตัวอย่างชุดแกน)

- QR1 Booking Fulfillment Rate (BFR) $\geq 99\%$
- QR2 Status Update Latency (SUL p95) ≤ 5 sec
- QR3 Manual Intervention Rate (MIR) $\leq 1\%$
- QR4 Confidence Indicator Accuracy (High-confidence fail $\leq 1\%$)
- QR5 Adoption Increase (Advance booking +30% ใน pilot) (ใช้ประกอบ Validation)

หมายเหตุ: QR5 เป็นพฤติกรรมผู้ใช้ จึงถือเป็น Validation-heavy

10.4 Verification Plan (Test Specification Sheet)

รูปแบบนี้ให้นักศึกษาทำ “1 หน้า ต่อ 1 QR”

ด้านล่างเป็นตัวอย่างละเอียดครบทุกช่อง

QR1 Verification — Booking Fulfillment Rate (BFR)

A) นิยาม Metric $BFR = (\# \text{ bookings ที่ check-in ได้สำเร็จ}) / (\# \text{ bookings ที่ถึงเวลาเริ่ม})$

B) Test Objective พิสูจน์ว่าระบบลด booking conflict และจองแล้วใช้งานได้จริง

C) Test Setup

- จำลองสภาพแวดล้อม 1 สาขา
- ทรัพยากร: ห้องประชุม 5 ห้อง, โต๊ะ 20 ที่, โปรเจคเตอร์ 2 ตัว
- สร้างผู้ใช้จำลอง 50 คน (mix: regular/premium)
- รวมทั้ง booking ล่วงหน้าและ walk-in

D) Test Procedure (Step-by-step)

1. สร้าง booking 500 รายการใน 7 วัน (สุ่มช่วงเวลา)
2. สร้าง event walk-in 50 รายการ (สุ่มชนบางช่วง)
3. ให้ผู้ใช้จำลองทำ check-in ตามเวลา
4. บันทึกทุกเคสที่ check-in ไม่ได้ พร้อมเหตุผล
5. คำนวณ BFR

E) Evidence to Collect (หลักฐาน)

- Log: booking_id, resource_id, start_time, checkin_status, failure_reason
- Report: BFR summary + list of failures

F) Pass/Fail Criteria

- **Pass:** $BFR \geq 99\%$
- **Fail:** $BFR < 99\%$ หรือมี failure ประเภท “resource unavailable despite confirmation” เกิน 0.5%

G) Typical Failure Diagnosis (สอนให้รู้ถ้าพังดูตรงไหน)

- ถ้า failure เป็น conflict → ปัญหา concurrency control
- ถ้า failure เป็น stale status → ปัญหา status update pipeline
- ถ้า failure เป็น walk-in override → ปัญหา policy enforcement

QR2 Verification — Status Update Latency (SUL)

A) นิยาม Metric SUL = เวลาที่ event เกิดจริง → เวลาที่ระบบแสดงสถานะใหม่สำเร็จ

B) Test Objective พิสูจน์ว่า “สถานะ real-time” จริง ไม่ใช่แค่ UI refresh

C) Test Setup

- สร้าง event update 300 เหตุการณ์:
 - check-in/out
 - cancel
 - maintenance toggle
 - no-show marking

D) Test Procedure

1. ทุก event ต้องสร้าง timestamp ที่ client (t_{client})
2. เมื่อระบบ commit สถานะใหม่ใน DB ให้บันทึก (t_{commit})
3. $Latency = t_{commit} - t_{client}$
4. คำนวณ p95

E) Evidence

- ตาราง latency ทุกเหตุการณ์ + histogram (optional)
- ค่าพิเศษ: max latency, failure events

F) Pass/Fail

- Pass: $p95 \leq 5 \text{ sec}$
- Fail: $p95 > 5 \text{ sec}$ หรือ $max > 30 \text{ sec}$ เกิน 1% ของเหตุการณ์

QR3 Verification — Manual Intervention Rate (MIR)

A) นิยาม Metric $MIR = (\# \text{ bookings ที่ต้อง staff/admin override}) / (\text{total bookings})$

B) Test Objective พิสูจน์ว่าระบบแก้สถานการณ์หลักแบบอัตโนมัติได้จริง

C) Test Setup

- ใช้ชุดข้อมูลเดียวกับ QR1
- เพิ่มสถานการณ์ผิดปกติ 30 เคส เช่น
 - user late
 - double booking attempt
 - payment fail then retry
 - dispute penalty

D) Test Procedure

1. บังคับให้ระบบเจอสถานการณ์ผิดปกติ
2. บันทึกว่ามีการกด “override” หรือไม่
3. สรุปจำนวน override / สาเหตุ
4. คำนวณ MIR

E) Evidence

- ticket log: override_type, who_override, reason
- MIR summary

F) Pass/Fail

- Pass: $MIR \leq 1\%$
- Fail: $MIR > 1\%$ หรือมี override ประเภทเดิมซ้ำเกิน 10 เคส (แปลว่าดีไซน์ยังไม่ครอบคลุม)

QR4 Verification — Confidence Indicator Accuracy

A) นิยาม Metric สำหรับ booking ที่ระบบระบุว่า **High confidence** อัตราการ “ล้มเหลว” ต้อง $\leq 1\%$

High-confidence fail rate = (# high-confidence bookings that fail) / (# high-confidence bookings)

B) Test Objective

พิสูจน์ว่าระบบ “ความมั่นใจสูง” ไม่หลอกลูกค้า (เพราะทำลาย trust)

C) Test Setup

- ทำ booking 300 รายการใน pilot
- ต้องมี high/medium/low กระจาย
- เก็บ predicted confidence ก่อนกดยืนยัน

D) Test Procedure

1. ก่อนสร้าง booking บันทึก predicted_confidence
2. หลังถึงเวลาเริ่ม booking ตรวจสอบผลจริง:
 - o success check-in
 - o fail due to conflict/unavailable
3. สรุป confusion matrix
4. คำนวณ high-confidence fail rate

E) Evidence

- predicted_confidence log
- outcome log
- confusion matrix

F) Pass/Fail

- **Pass:** High-confidence fail $\leq 1\%$
- **Fail:** $> 1\%$ (ถือว่า critical failure)

10.5 Validation Plan (User Validation)

Validation ต้อง “กลับไปหา stakeholder” เพื่อพิสูจน์ว่าเราแก้ pain point ที่แท้จริงได้

Validation-1: Premium Member Scenario-based Evaluation

Objective

พิสูจน์ว่า “ผู้ใช้รู้สึกเชื่อมั่นขึ้น” และ “ไม่ต้องมี plan B”

Method

- ให้ Premium Member 3 คนทำภารกิจจริง:
 1. จองห้องประชุมล่วงหน้า
 2. มาถึงแล้ว check-in
 3. พบสถานการณ์ walk-in / no-show case (จำลอง)

Metrics

- Task completion rate (%)
- Task time (min)
- Reported confidence score (1–5)
- times asking staff for help

Pass Criteria (ตัวอย่าง)

- $\geq 90\%$ tasks สำเร็จ
- เวลาเฉลี่ยในการจองลดลง $\geq 30\%$
- Confidence score $\geq 4/5$
- การเรียก staff ช่วยลดลง $\geq 50\%$

Validation-2: Staff Workflow Validation

Objective

พิสูจน์ว่า staff workload ลดจริง และ process ไม่ยุ่งขึ้น

Metrics

- เวลาเฉลี่ยรับมือเคส conflict (ก่อน/หลัง)
- manual override
- satisfaction score ของ staff

10.6 Final Evaluation Report (รูปแบบสรุปผล)

ตารางนี้คือสิ่งที่ควรอยู่ในบท “ผลการทดลอง” ของรายงาน

Requirement (QR)	Target	Result	Pass/Fail	Evidence
QR1 BFR	$\geq 99\%$	99.3%	PASS	booking logs + failure list
QR2 SUL p95	≤ 5 sec	3.8 sec	PASS	latency table
QR3 MIR	$\leq 1\%$	0.6%	PASS	override tickets
QR4 High-conf fail	$\leq 1\%$	0.9%	PASS	confusion matrix
Validation confidence	$\geq 4/5$	4.3/5	PASS	survey + interview notes

หมายเหตุ: ตัวเลขผลลัพธ์ในตัวอย่างนี้ “สมมติ” เพื่อแสดงรูปแบบการเขียนรายงาน
นักศึกษาต้องแทนด้วยผลจริงจากการทดสอบ

10.7 ถ้า Fail ต้องทำอะไร (สำคัญมาก)

ในรายงานที่ดี ไม่ใช่ต้อง PASS ทุกตัว แต่ต้อง “รู้ว่าพังเพราะอะไร และปรับอะไร”

ตัวอย่าง

- ถ้า QR2 fail (latency สูง)
 - วิเคราะห์ bottleneck: DB write / queue backlog / notification coupling
 - แนวทางแก้: async event processing, index tuning, batching update
- ถ้า QR4 fail (confidence หลอก)
 - ลด scope เป็น rule-based
 - เพิ่ม safety: ถ้า data น้อย $\rightarrow$ ไม่อนุญาตแสดง high confidence

Output ของขั้นตอน V&V

1. Test Plan ครบทุก QR
2. Test Cases แบบ step-by-step
3. เกณฑ์ Pass/Fail ชัด
4. หลักฐานที่ต้องเก็บ (logs / tables / metrics)
5. Final Evaluation Summary
6. Lessons Learned + Improvement Plan

บทที่ 4 ผลการดำเนินงานและการประเมินผล (Results & Evaluation)

4.1 วัตถุประสงค์ของบทนี้

บทนี้มีวัตถุประสงค์เพื่อ:

1. รายงาน ผลการทดสอบเชิงวิศวกรรม (Verification) ตาม Quantifiable Requirements (QRs)
2. ประเมินว่า ระบบตอบสนองความต้องการของผู้ใช้จริง (Validation) หรือไม่
3. วิเคราะห์ผลลัพธ์ที่ได้ เทียบกับเป้าหมายของโครงการ
4. สรุปข้อค้นพบและประเด็นที่ต้องปรับปรุงในอนาคต

4.2 สภาพแวดล้อมและเงื่อนไขการทดสอบ (Test Environment)

4.2.1 สภาพแวดล้อมระบบ

- รูปแบบการทดสอบ: ระบบต้นแบบ (Prototype)
- ขอบเขต: 1 สาขาจำลองของ coworking space
- ทรัพยากร:
 - ห้องประชุม 5 ห้อง
 - โต๊ะทำงาน 20 ที่
 - อุปกรณ์เสริม 3 ประเภท
- ผู้ใช้จำลอง: 50 คน
(Regular Member 35 คน, Premium Member 15 คน)

4.2.2 เงื่อนไขการใช้งาน

- ระยะเวลาทดสอบ: 7 วัน (Pilot Period)
- รูปแบบการจอง:
 - การจองล่วงหน้า
 - Walk-in
- เหตุการณ์พิเศษ:
 - No-show
 - การยกเลิกหน้างาน
 - ความล่าช้าในการ check-in

4.3 ผลการทดสอบเชิง Verification (QR-based Verification Results)

4.3.1 QR1: Booking Fulfillment Rate (BFR)

เป้าหมาย:

$BFR \geq 99\%$

ผลการทดสอบ:

- จำนวน booking ทั้งหมด: 520 รายการ

- จำนวน booking ที่ check-in สำเร็จ: 516 รายการ

[

$$\text{BFR} = \frac{516}{520} \times 100 = 99.23\%$$

]

ผลการประเมิน:

- ผ่าน (PASS)

การวิเคราะห์:

เหตุการณ์ที่ไม่สามารถใช้งานได้จริง 4 กรณีเกิดจาก:

- การยกเลิกทรัพยากรเพื่อ maintenance แบบฉุกเฉิน
- ไม่พบ booking conflict จากการจองซ้ำซ้อน

4.3.2 QR2: Status Update Latency (p95)

เป้าหมาย:

$$p95 \leq 5 \text{ วินาที}$$

ผลการทดสอบ:

- ค่าเฉลี่ย latency: 2.1 วินาที
- ค่า p95 latency: 3.8 วินาที
- ค่า latency สูงสุด: 8.6 วินาที (1 เหตุการณ์)

ผลการประเมิน:

- ผ่าน (PASS)

การวิเคราะห์:

เหตุการณ์ latency สูงสุดเกิดจาก notification queue backlog ซึ่งไม่กระทบต่อค่า p95 และไม่ทำให้สถานะผิดพลาด

4.3.3 QR3: Manual Intervention Rate (MIR)

เป้าหมาย:

$$\text{MIR} \leq 1\%$$

ผลการทดสอบ:

- จำนวน booking ทั้งหมด: 520
- จำนวนกรณีที่ต้อง staff/admin override: 3 กรณี

[

$$\text{MIR} = \frac{3}{520} \times 100 = 0.58\%$$

]

ผลการประเมิน:

- ผ่าน (PASS)

การวิเคราะห์:

กรณี override เกิดจาก:

- ลูกค้านำร่องขอผ่อนผันค่าปรับ (exception policy)
- ไม่พบ override จาก booking conflict

4.3.4 QR4: Confidence Indicator Accuracy

เป้าหมาย:

High-confidence fail rate $\leq 1\%$

ผลการทดสอบ:

Confidence Level จำนวน Booking ล้มเหลว

High	210	2
Medium	180	6
Low	130	11

[
High\text{-}confidence\ fail\ rate = $\frac{2}{210} \times 100 = 0.95\%$
]

ผลการประเมิน:

- ผ่าน (PASS)

การวิเคราะห์:

ความผิดพลาดในกลุ่ม High-confidence เกิดจาก:

- การยกเลิกทรัพยากรฉุกเฉิน (maintenance)
- ไม่พบกรณี conflict จากการคาดการณ์ผิด

4.4 สรุปผล Verification โดยรวม

QR	เป้าหมาย	ผลลัพธ์	สถานะ
QR1 Booking Fulfillment	$\geq 99\%$	99.23%	PASS
QR2 Status Latency p95	$\leq 5 \text{ sec}$	3.8 sec	PASS
QR3 Manual Intervention	$\leq 1\%$	0.58%	PASS
QR4 Confidence Accuracy	$\leq 1\%$	0.95%	PASS

จากผลการทดสอบ ระบบสามารถผ่าน Quantifiable Requirements ทุกข้อที่กำหนดไว้ในขั้นตอนออกแบบ

4.5 ผลการ Validation กับผู้ใช้งานจริง (User Validation)

4.5.1 Premium Member Validation

กลุ่มตัวอย่าง: Premium Member 5 คน

ภารกิจ: จองห้องประชุมล่วงหน้าและใช้งานจริง

ตัวชี้วัด	ก่อนใช้ระบบ	หลังใช้ระบบ
เวลาเฉลี่ยในการจอง	4.5 นาที	2.8 นาที
จำนวนครั้งขอความช่วยเหลือ staff	3.2 ครั้ง/สัปดาห์	1.1 ครั้ง/สัปดาห์
คะแนนความมั่นใจ (1-5)	3.1	4.3

ผลการประเมิน:

- ผู้ใช้รายงานว่าระบบ “เชื่อถือได้มากขึ้น”
- ไม่จำเป็นต้องเตรียมแผนสำรองเมื่อจองล่วงหน้า

4.5.2 Staff Workflow Validation

ตัวชี้วัด	ก่อน	หลัง
เวลาเฉลี่ยจัดการ conflict	12 นาที	4 นาที
จำนวน manual override	สูง	ต่ำมาก
ความพึงพอใจ staff (1-5)	3.4	4.2

ผลการประเมิน:

ระบบช่วยลดภาระงานเชิงปฏิบัติการของพนักงานอย่างมีนัยสำคัญ

4.6 การตอบโภทย์ Voice of the Customer (VoC Traceback)

VoC หลัก	หลักฐานจาก Results
“จองแล้วต้องได้ใช้จริง”	BFR = 99.23%
“สถานะต้อง real-time”	p95 latency = 3.8 sec
“ไม่อยากเครียด/ไม่อยากมี plan B”	Confidence score ↑
“ไม่อยากต้องหาพนักงานบ่อย”	MIR = 0.58%

แสดงให้เห็นว่าโซลูชันที่พัฒนาขึ้นสามารถตอบ pain point หลักของลูกค้าได้จริง

4.7 ข้อจำกัดของผลการทดลอง

แม้ระบบจะผ่าน QR ทุกข้อ แต่ยังมีข้อจำกัดดังนี้:

1. ระยะเวลา pilot ค่อนข้างสั้น (7 วัน)
2. ปริมาณข้อมูลสำหรับ prediction ยังจำกัด
3. การทดสอบทำในสภาพแวดล้อมจำลอง ไม่ใช่การใช้งานจริงเต็มรูปแบบ

4.8 สรุปบท Results

จากผลการทดสอบและการประเมิน:

- ระบบสามารถผ่าน Quantifiable Requirements ทุกข้อ
- ผู้ใช้รายงานความเชื่อมั่นในการจองล่วงหน้าเพิ่มขึ้น
- ภาระงานของพนักงานลดลงอย่างชัดเจน
- แนวคิด Prediction-based Reservation Control เหมาะสมกับบริบท coworking space

ผลลัพธ์เหล่านี้ยืนยันว่า

โครงการนี้ไม่เพียง “สร้างระบบได้” แต่ “สร้างระบบที่ตอบโจทย์ผู้ใช้จริง”

บทที่ 5 บทสรุป อภิปรายผล และข้อเสนอแนะ (Conclusion & Future Work)

5.1 สรุปภาพรวมของโครงการ (Project Summary)

โครงการ Capstone Design Project นี้มีวัตถุประสงค์เพื่อออกแบบและพัฒนาระบบบริหารจัดการ coworking space ที่สามารถแก้ปัญหาหลักของการจองพื้นที่และทรัพยากรซึ่งพบได้บ่อยในสถานการณ์ใช้งานจริง เช่น

- การจองซ้ำซ้อน (booking conflict)
- การไม่มาใช้งานตามเวลาที่จอง (no-show)
- สถานะทรัพยากรไม่ตรงกับความเป็นจริง
- การพึ่งพาการแก้ไขแบบ manual จากพนักงาน

ทีมผู้จัดทำได้เลือกใช้แนวคิด **Prediction-based Reservation Control (PRC)** เพื่อควบคุมและป้องกันปัญหาเชิงรุก โดยใช้ข้อมูลพฤติกรรมการใช้งานมาประเมินความเสี่ยงของการจอง ก่อนที่ปัญหาจะเกิดขึ้นจริง

5.2 การบรรลุวัตถุประสงค์ของโครงการ (Achievement of Objectives)

จากผลการดำเนินงานและการทดสอบในบทที่ 4 สามารถสรุปการบรรลุวัตถุประสงค์ของโครงการได้ดังนี้

5.2.1 ด้านวิศวกรรมระบบ (Engineering Objectives)

โครงการสามารถ:

- แปลง Voice of the Customer (VoC) ไปเป็น Quantifiable Requirements (QRs) ที่วัดผลได้
- ออกแบบระบบโดยใช้หลัก QFD, Engineering Characteristics และ System Architecture
- พัฒนาระบบที่ผ่านการทดสอบตาม QR ทุกข้อที่กำหนดไว้

ผลการ Verification แสดงให้เห็นว่า:

- อัตราการจองที่ใช้งานได้จริง $\geq 99\%$
- ความหน่วงของการอัปเดตสถานะทรัพยากรอยู่ในเกณฑ์ที่ยอมรับได้
- อัตราการแก้ไขปัญหาแบบ manual ต่ำกว่า 1%

5.2.2 ด้านผู้ใช้งานและธุรกิจ (User & Business Objectives)

จากการ Validation กับผู้ใช้งานตัวอย่าง:

- ผู้ใช้มีความเชื่อมั่นในการจองล่วงหน้าเพิ่มขึ้น
- ภาระงานของพนักงานลดลงอย่างชัดเจน
- กระบวนการใช้งานมีความเป็นระบบและโปร่งใสมากขึ้น

แสดงให้เห็นว่าโครงการสามารถตอบโจทย์ **ปัญหาที่แท้จริงของผู้ใช้ (Real Pain Points)** ไม่ใช่เพียงการพัฒนาตามสมมติฐานของผู้พัฒนา

5.3 การอภิปรายผลเชิงวิศวกรรม (Engineering Discussion)

5.3.1 คุณค่าของแนวคิด Prediction-based Reservation Control

แนวคิด PRC มีข้อดีเชิงวิศวกรรมที่สำคัญ ได้แก่:

- เปลี่ยนจากการแก้ปัญหาเชิงรับ (reactive) เป็นการป้องกันเชิงรุก (proactive)
- ลดการพึ่งพาการตัดสินใจของมนุษย์ในสถานการณ์ซ้ำ ๆ
- สนับสนุนการออกแบบระบบที่ “เชื่อถือได้” (Trustworthy System)

ผลการทดสอบแสดงให้เห็นว่า แม้จะใช้เพียง rule-based prediction ในระดับต้น ก็สามารถลดปัญหา no-show และ booking conflict ได้อย่างมีนัยสำคัญ

5.3.2 การเชื่อมโยงกระบวนการ Capstone อย่างเป็นระบบ

โครงงานนี้แสดงให้เห็นความสำคัญของการทำ Capstone ตามกระบวนการวิศวกรรมอย่างครบถ้วน ได้แก่:

- Stakeholder Analysis → VoC
- VoC → Problem Definition
- Problem → Quantifiable Requirements
- QR → QFD & ECs
- ECs → Concept Selection
- Design → Verification & Validation

การรักษา **Traceability** ระหว่างแต่ละขั้น ช่วยให้การตัดสินใจเชิงออกแบบมีเหตุผลรองรับ และสามารถอธิบายต่อกรรมการหรือผู้มีส่วนได้ส่วนเสียได้อย่างชัดเจน

5.4 ข้อจำกัดของโครงงาน (Limitations)

แม้โครงงานจะบรรลุเป้าหมายที่ตั้งไว้ แต่ยังมีข้อจำกัดที่ควรพิจารณา ได้แก่:

1. การทดสอบทำในสภาพแวดล้อมจำลอง (pilot) ยังไม่ครอบคลุมพฤติกรรมผู้ใช้ในระยะยาว
2. โมเดลการคาดการณ์ใช้ข้อมูลในช่วงเวลาจำกัด อาจยังไม่สะท้อนฤดูกาลหรือแนวโน้มระยะยาว
3. ระบบยังไม่ครอบคลุมการเชื่อมต่อกับระบบภายนอกจริง เช่น ระบบบัญชีหรือ CRM ขององค์กรขนาดใหญ่

ข้อจำกัดเหล่านี้เป็นโอกาสในการพัฒนาโครงงานต่อไปในอนาคต

5.5 ข้อเสนอแนะและแนวทางพัฒนาต่อ (Future Work)

5.5.1 การพัฒนาด้านเทคนิค

- เพิ่มการเรียนรู้เชิงสถิติหรือ Machine Learning เพื่อปรับปรุงความแม่นยำของการประเมินความเสี่ยง
- ปรับปรุงสถาปัตยกรรมเพื่อรองรับหลายสาขาพร้อมกัน
- เพิ่มระบบ monitoring และ alert สำหรับ latency และ error

5.5.2 การพัฒนาด้านการใช้งาน

- ปรับปรุง UI/UX สำหรับการแสดง confidence indicator ให้เข้าใจง่ายและไม่สร้างความสับสน
- เพิ่มกลไก feedback จากผู้ใช้เพื่อปรับ policy แบบ dynamic
- รองรับรูปแบบสมาชิกและแพ็คเกจที่หลากหลายขึ้น

5.5.3 การต่อยอดเชิงธุรกิจ

- นำข้อมูลเชิงพฤติกรรมไปใช้วิเคราะห์แนวโน้มรายได้
- สนับสนุนการทำ dynamic pricing ตามช่วงเวลาใช้งาน
- เชื่อมต่อกับระบบ partner หรือ ecosystem ภายนอก

5.6 บทเรียนที่ได้รับจาก Capstone Design Project (Lessons Learned)

จากการดำเนินโครงการนี้ ผู้จัดทำได้เรียนรู้ว่า:

1. ปัญหาที่แท้จริงมักซ่อนอยู่หลังคำบ่นของผู้ใช้
2. การออกแบบเชิงวิศวกรรมต้องเริ่มจากความเข้าใจ ไม่ใช่เทคโนโลยี
3. การวัดผลและการทดสอบมีความสำคัญเทียบเท่าการพัฒนา
4. Capstone ที่ดีต้องสามารถ “อธิบายเหตุผลของการตัดสินใจได้”

5.7 สรุปปิดท้าย

โครงการ Capstone Design Project นี้ แสดงให้เห็นถึงการประยุกต์ใช้กระบวนการออกแบบเชิงวิศวกรรม ตั้งแต่การค้นหาคำต้องการของผู้ใช้ ไปจนถึงการพิสูจน์ผลลัพธ์ด้วยการทดสอบอย่างเป็นระบบ ผลลัพธ์ของโครงการไม่เพียงแต่ตอบโจทย์ทางเทคนิค แต่ยังสะท้อนการเรียนรู้และการเติบโตของผู้จัดทำ ในฐานะวิศวกรที่สามารถออกแบบระบบเพื่อแก้ปัญหาในโลกจริงได้อย่างมีความรับผิดชอบ